

2.4 中文分词与条件随机场

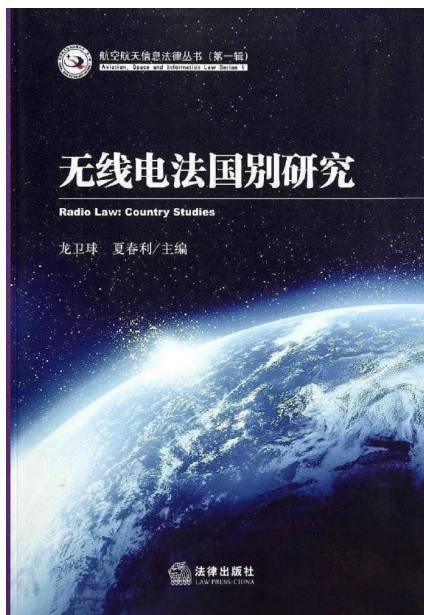
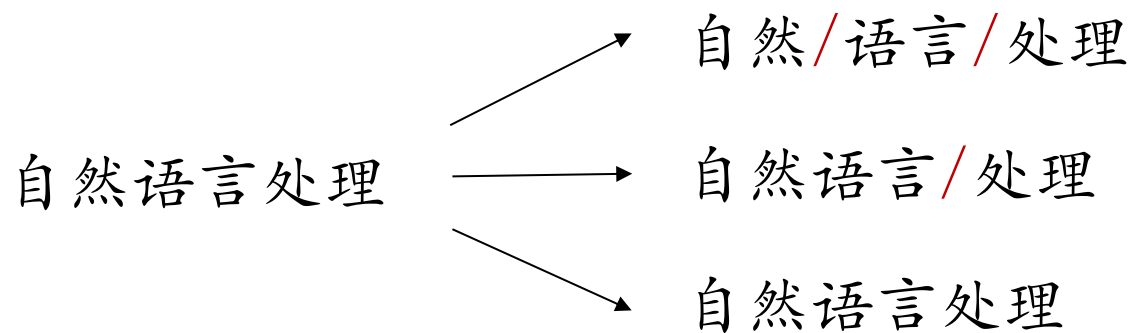
陈露 & 王瑞

上海交通大学计算机科学与工程系

2023年秋

- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标



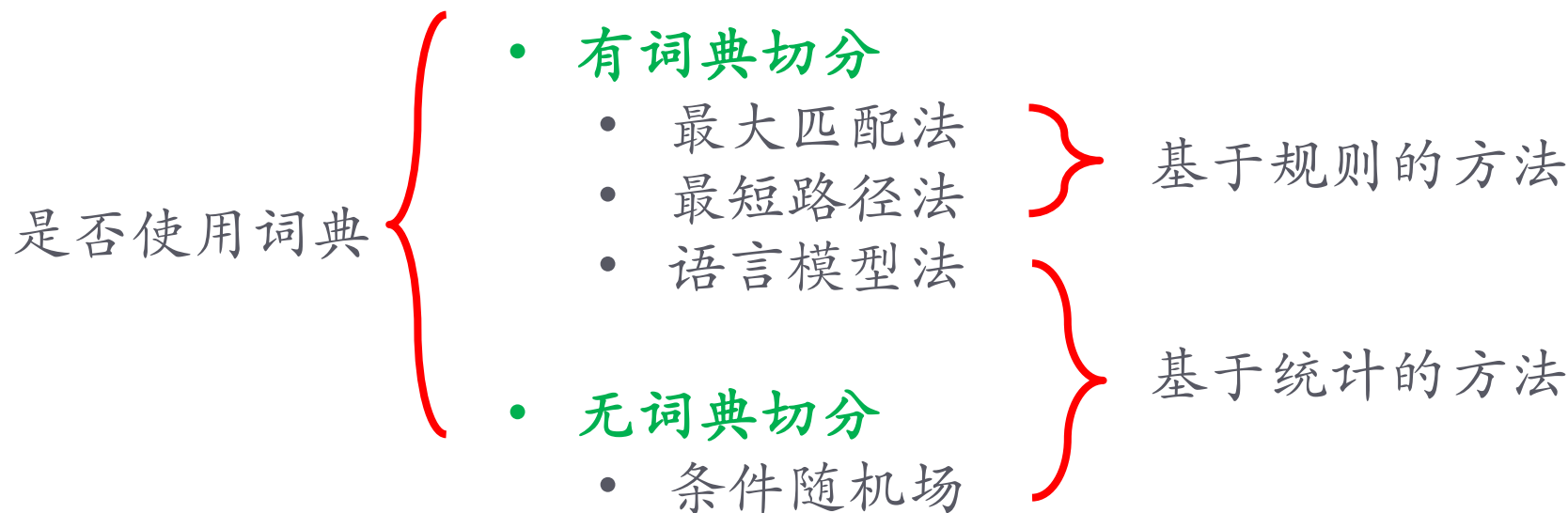
中文分词的重要性

- ▶ 自动分词是汉语句子分析的基础
- ▶ 分词具有广泛的应用
 - ▶ 词频统计
 - ▶ 词典编撰
 - ▶ 主题分类
- ▶ 分词直接影响了后续文本处理算法所用的特征
 - ▶ 词包法 (bag-of-words) 的文本分类
 - ▶ 句法解析
 - ▶

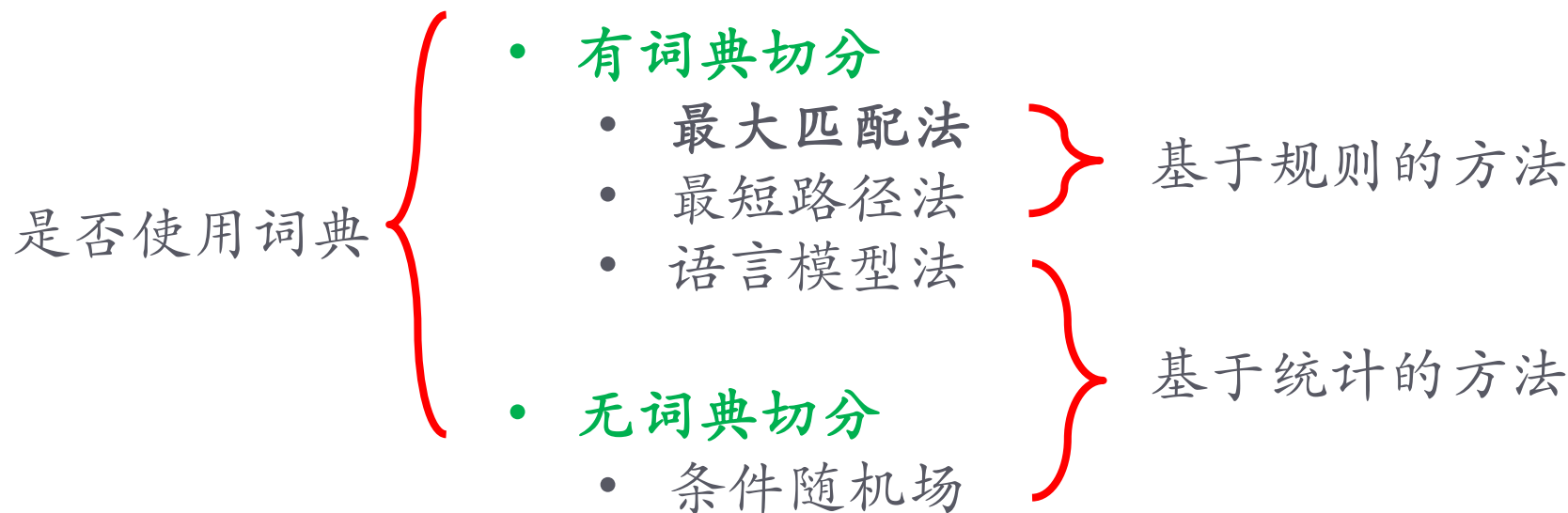


- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

中文分词的基本算法



中文分词的基本算法



中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

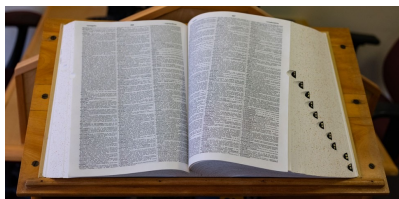
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 是 研 究 生 物 化 学 的



词典:

最长单词: 6字

最短单词: 1字

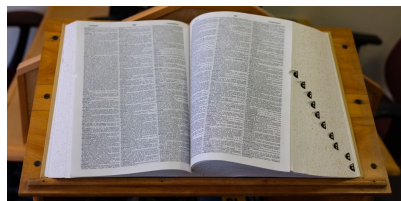
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 研 究 生 物 化 学 的



词典:

最长单词: 6字

最短单词: 1字

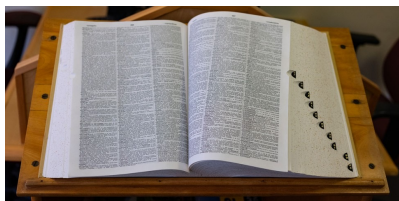
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 / 研 究 生 物 化 学 的



词典:

最长单词: 6字

最短单词: 1字



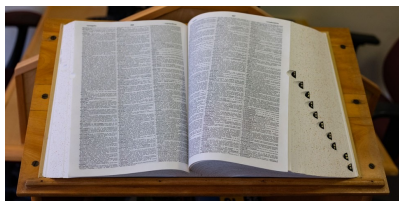
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 / 研 究 生 / 物 化 学 的



词典:

最长单词: 6字

最短单词: 1字

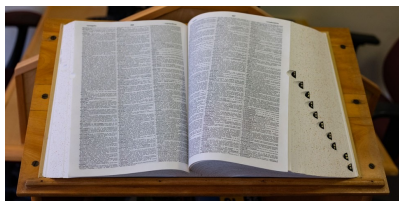
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 / 研 究 生 / 物 化 / 学 的



词典:

最长单词: 6字

最短单词: 1字

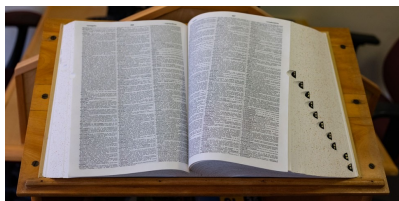
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 / 研 究 生 / 物 化 / 学 的



词典:

最长单词: 6字

最短单词: 1字

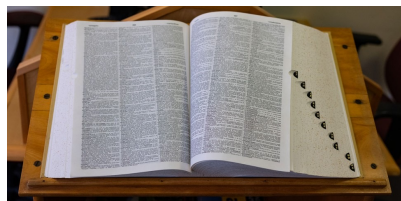
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 / 研 究 生 / 物 化 / 学 / 的



词典:

最长单词: 6字

最短单词: 1字

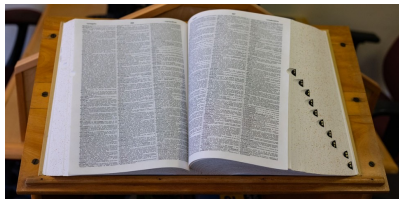
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 是 研 究 生 物 化 学 的



词典:

最长单词: 6字

最短单词: 1字

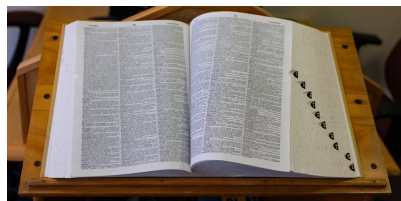
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 是 研 究 生 物 化 学 / 的



词典:

最长单词: 6字

最短单词: 1字

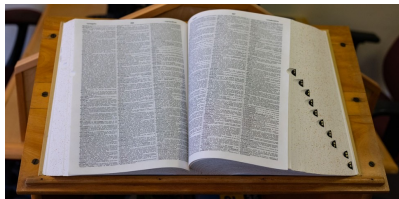
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 是 研 究 生 物 / 化 学 / 的



词典:

最长单词: 6字

最短单词: 1字

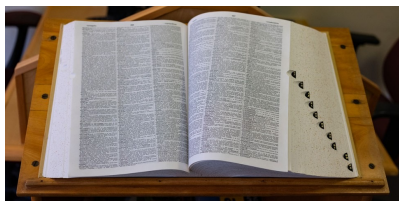
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 是 研 究 / 生 物 / 化 学 / 的



词典:

最长单词: 6字

最短单词: 1字

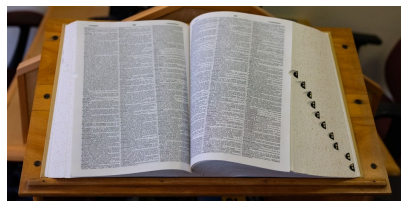
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 是 / 研 究 / 生 物 / 化 学 / 的



词典:

最长单词: 6字

最短单词: 1字

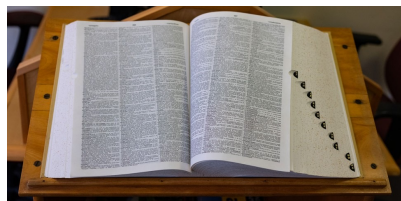
中文分词的基本算法：最大匹配法

最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

他 / 是 / 研 究 / 生 物 / 化 学 / 的



词典:

最长单词: 6字

最短单词: 1字

中文分词的基本算法：最大匹配法

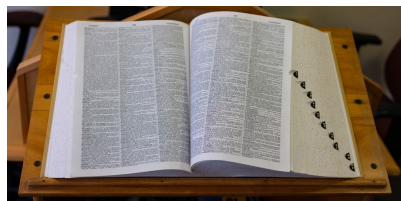
最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

逆向最大匹配算法结果：

他 / 是 / 研 究 / 生 物 / 化 学 / 的



正向最大匹配算法结果：

他 / 是 / 研 究 生 / 物 / 化 学 / 的

词典：

最长单词：6字

最短单词：1字

中文分词的基本算法：最大匹配法

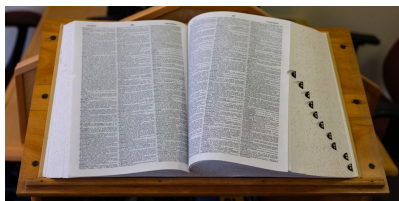
最大匹配算法 (Maximum Matching)

正向最大匹配算法 (Forward MM, FMM)

逆向最大匹配算法 (Backward MM, BMM)

优点：

- 程序简单易行，开发周期短
- 仅需要很少的语言资源（词表），不需要任何词法、句法、语义资源



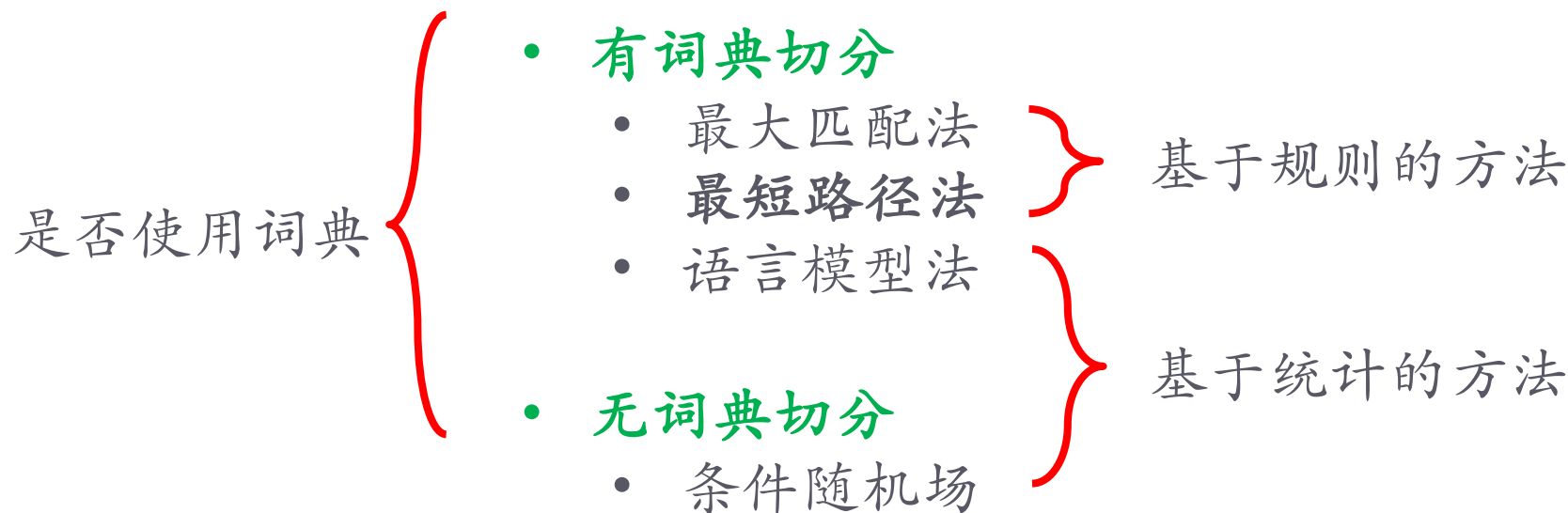
缺点：

- 歧义消解的能力差

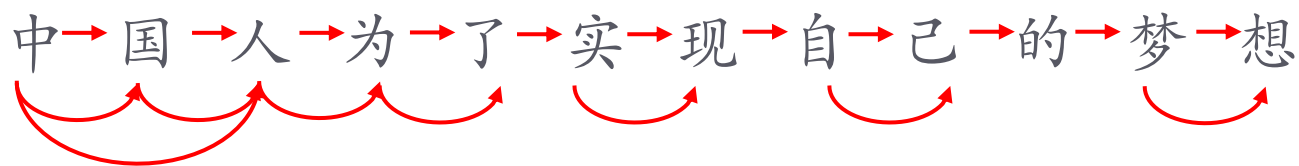
词典：

最长单词：6字

最短单词：1字



中文分词的基本算法：最短路径法

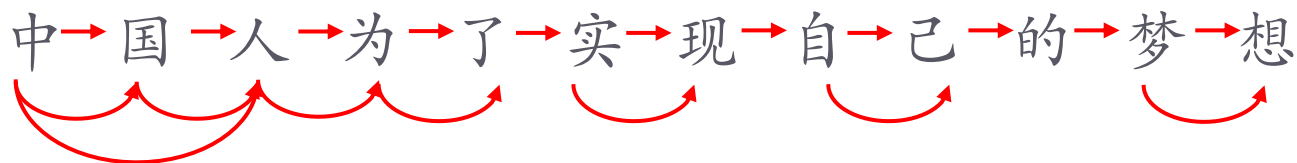


从以上所有路径中，选择路径最短的
(词数最少的)作为最终分词结果



中 国 人 / 为 了 / 实 现 / 自 己 / 的 / 梦 想

中文分词的基本算法：最短路径法

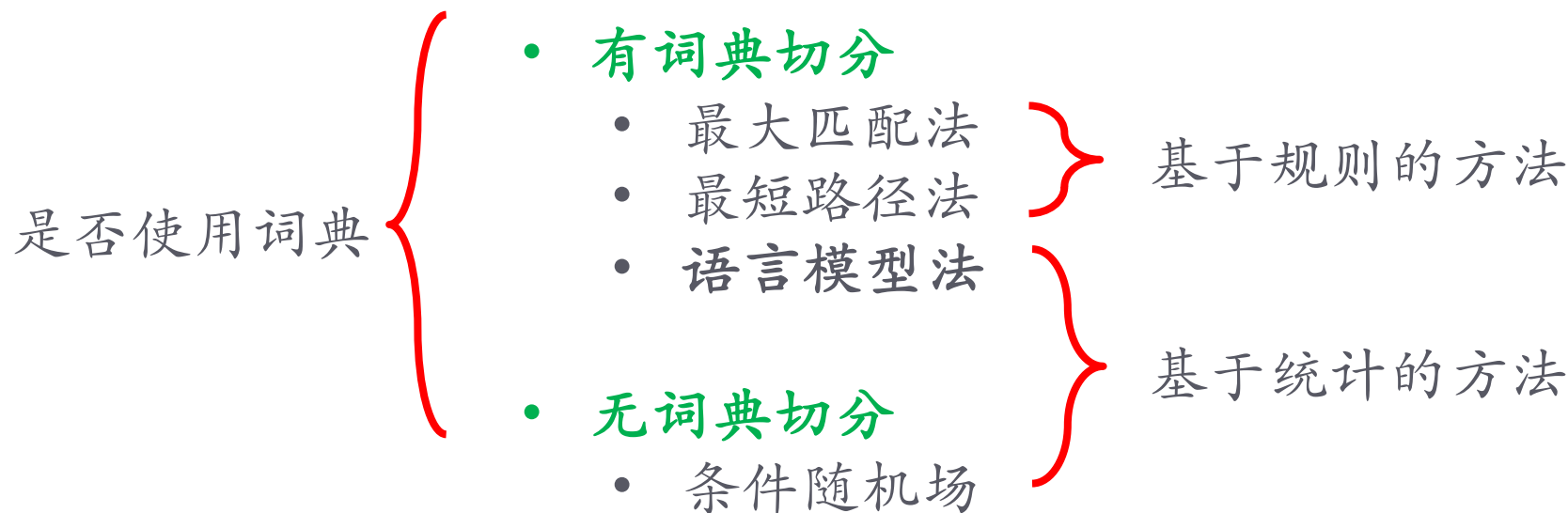


优点：

- 切分原则明确，符合汉语自身规律
- 需要的语言资源（词表）也不多

缺点：

- 对许多歧义字段难以区分，最短路径有多条时，选择最终的输出结果缺乏应有的标准
- 字符串长度较大和选取的最短路径数增大时，长度相同的路径数急剧增加，选择最终正确的结果困难越来越大



中文分词的基本算法：语言模型法

设句子 $S = \{s_1, s_2, \dots, s_N\}$ 可切分为 K 个单词 $W = \{w_1, w_2, \dots, w_K\}$ ，则此任务为找到最优的 W ，即 W^* ：

$$\begin{aligned} W^* &= \operatorname{argmax}_W P(W|S) \\ &= \operatorname{argmax}_W P(W) \boxed{P(S|W)} \\ &= \operatorname{argmax}_W P(W) \quad = 1 \end{aligned}$$

对于给定的 $P(W)$ ，其概率由预先训练好的语言模型给出。

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

穷举所有可能的 W ，分别计算其对应的 $P(W)$ ，挑出最大的即为 W^* 。

他是研究生物化学的

他 / 是 / 研 究 / 生 物 / 化 学 / 的 ✓

他 / 是 / 研 究 生 / 物 / 化 学 / 的 ✗

他 / 是 / 研 究 生 / 物 化 / 学 / 的 ✗

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

束搜索 (beam search)：当穷举所有可能的 W 变得极多而无法一一计算时，利用束搜索从左至右地确定分词方案 W^* 。

他是研究生物化学的

他 / 是

他 / 是 研

他 / 是 研 究

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

束搜索 (beam search)：当穷举所有可能的 W 变得极多而无法一一计算时，利用束搜索从左至右地确定分词方案 W^* 。

他是研究生物化学的				
他 / 是 / 研				×
他 / 是 / 研 究				✓
他 / 是	他 / 是 / 研 究 生			✓
	他 / 是 研 / 究			×
他 / 是 研	他 / 是 研 / 究 生			×
	他 / 是 研 / 究 生 物			×
他 / 是 研 究	他 / 是 研 究 / 生			×
	他 / 是 研 究 / 生 物			✓
	他 / 是 研 究 / 生 物 化			×

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

束搜索 (beam search)：当穷举所有可能的 W 变得极多而无法一一计算时，利用束搜索从左至右地确定分词方案 W^* 。

他是研究生物化学的

	他 / 是 / 研 究 / 生	×
他 / 是 / 研 究	他 / 是 / 研 究 / 生 物	✓
	他 / 是 / 研 究 / 生 物 化	✓
	他 / 是 / 研 究 生 / 物	×
他 / 是 / 研 究 生	他 / 是 / 研 究 生 / 物 化	✓
	他 / 是 / 研 究 生 / 物 化 学	×
	他 / 是 研 究 / 生 物 / 化	×
他 / 是 研 究 / 生 物	他 / 是 研 究 / 生 物 / 化 学	×
	他 / 是 研 究 / 生 物 / 化 学 的	×

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

束搜索 (beam search)：当穷举所有可能的 W 变得极多而无法一一计算时，利用束搜索从左至右地确定分词方案 W^* 。

他是研究生物化学的

	他 / 是 / 研 究 / 生 物 / 化	×
他 / 是 / 研 究 / 生 物	他 / 是 / 研 究 / 生 物 / 化 学	✓
	他 / 是 / 研 究 / 生 物 / 化 学 的	✓
	他 / 是 / 研 究 / 生 物 化 / 学	✓
他 / 是 / 研 究 / 生 物 化	他 / 是 / 研 究 / 生 物 化 / 学 的	×
	∅	×
	他 / 是 / 研 究 生 / 物 化 / 学	×
他 / 是 / 研 究 生 / 物 化	他 / 是 / 研 究 生 / 物 化 / 学 的	×
	∅	×

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

束搜索 (beam search)：当穷举所有可能的 W 变得极多而无法一一计算时，利用束搜索从左至右地确定分词方案 W^* 。

他是研究生物化学的

他 / 是 / 研 究 / 生 物 / 化 学

他 / 是 / 研 究 / 生 物 / 化 学 / 的 ✓

∅

×

∅

×

∅

×

他 / 是 / 研 究 / 生 物 / 化 学 的

∅

×

∅

×

他 / 是 / 研 究 / 生 物 化 / 学

他 / 是 / 研 究 / 生 物 化 / 学 / 的 ✓

∅

×

∅

×

✓

中文分词的基本算法：语言模型法

计算 $\operatorname{argmax}_W(\cdot)$ 的方法：

$$W^* = \operatorname{argmax}_W P(W|S)$$

束搜索 (beam search)：当穷举所有可能的 W 变得极多而无法一一计算时，利用束搜索从左至右地确定分词方案 W^* 。

无法保证找到全局最优解，但是可以在可接受的计算量下达到可接受的次优解

中文分词的基本算法：语言模型法

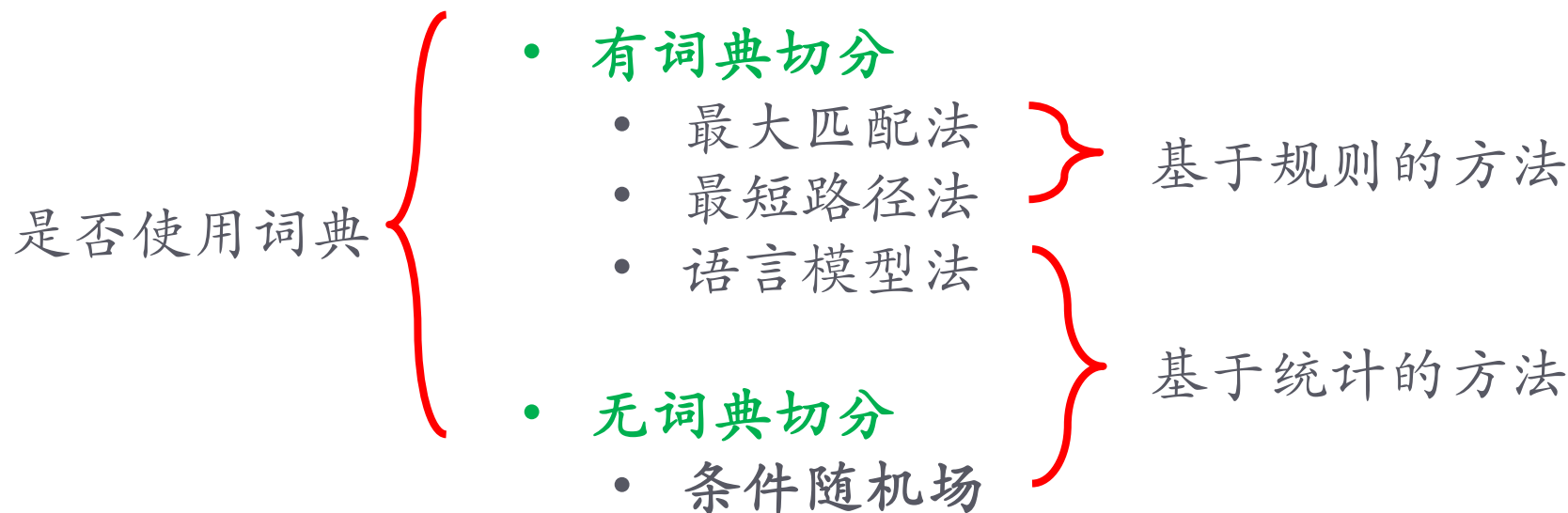
$$W^* = \operatorname{argmax}_W P(W|S)$$

优点：

- 减少了很多手工标注的工作
- 在训练语料规模足够大和覆盖领域足够多时，可以获得较高的切分正确率

缺点：

- 训练语料的规模和覆盖领域不好把握
- 计算量较大



中文分词的基本算法：条件随机场的问题设定

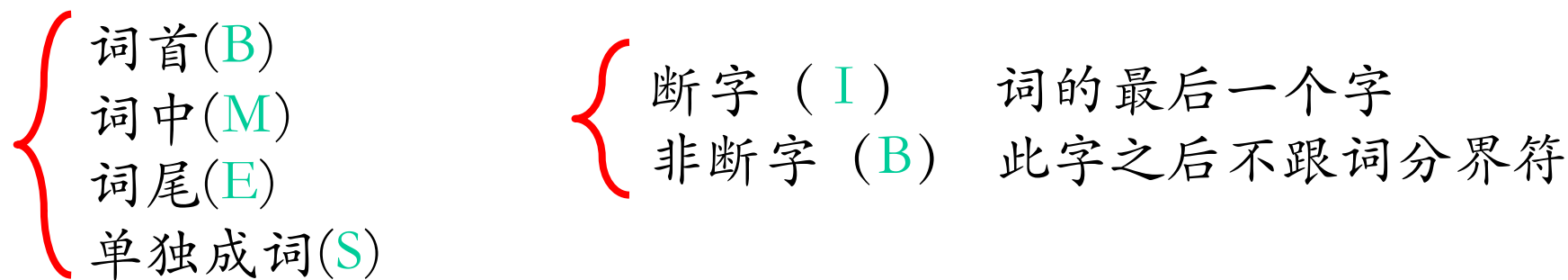
将分词看作是**序列标注**问题，依次对序列中每个字赋予一个标签，再根据标签合并同属于一个单词的字符：

- 词首(B)
- 词中(M)
- 词尾(E)
- 单独成词(S)

	中	国	人	/	为	了	/	实	现	/	自	己	/	的	/	梦	想
B	✓				✓			✓			✓					✓	
M		✓															
E			✓			✓			✓			✓					✓
S													✓				
	中	国	人	为	了	实	现	自	己	的	梦	想					

中文分词的基本算法：条件随机场的问题设定

将分词看作是序列标注问题，依次对序列中每个字赋予一个标签，再根据标签合并同属于一个单词的字符：



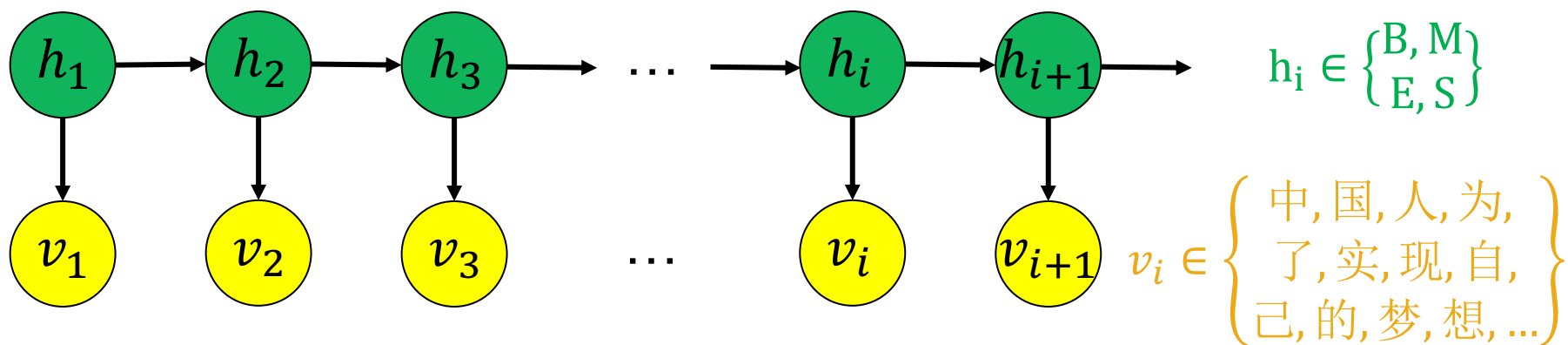
中 国 人 / 为 了 / 实 现 / 自 己 / 的 / 梦 想

B ✓ ✓ ✓ ✓ ✓ ✓

I ✓ ✓ ✓ ✓ ✓

中 国 人 为 了 实 现 自 己 的 梦 想

中文分词的基本算法：HMM



中 国 人 / 为 了 / 实 现 / 自 己 / 的 / 梦 想

B
M
E
S

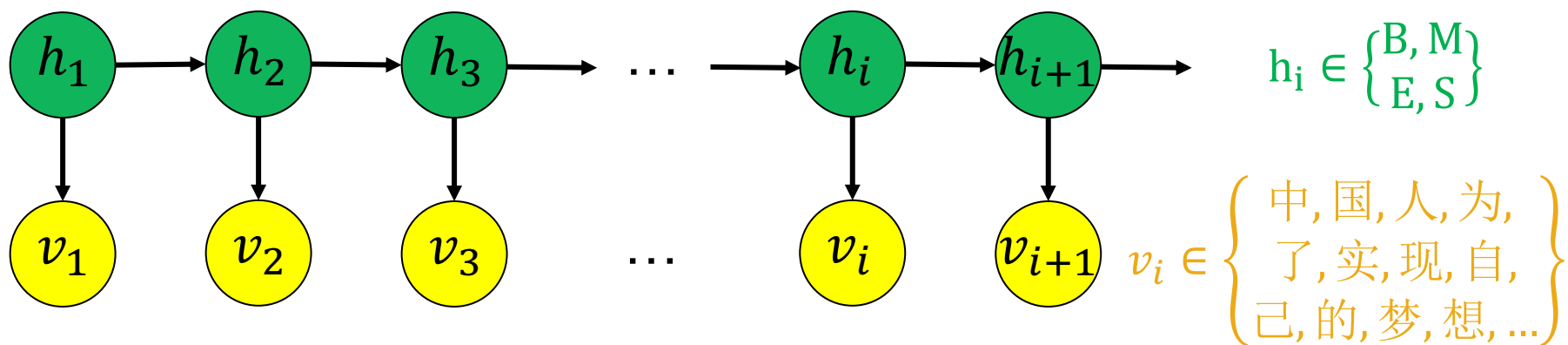
✓ ✓ ✓ ✓ ✓

✓ ✓ ✓ ✓ ✓

 ✓ ✓ ✓ ✓ ✓

中 国 人 为 了 实 现 自 己 的 梦 想

中文分词的基本算法：HMM

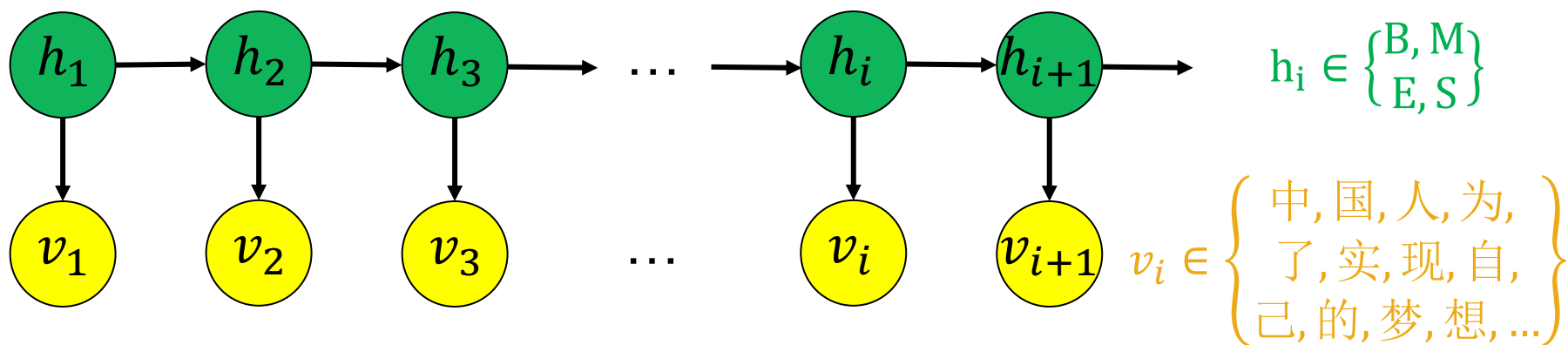


$$\operatorname{argmax}_{\underline{h}} P(\underline{h} | \underline{v}) = \operatorname{argmax}_{\underline{h}} P(\underline{h}, \underline{v})$$

$$= \operatorname{argmax}_{\underline{h}} (P(\underline{v} | \underline{h}) \cdot P(\underline{h}))$$

$$= \operatorname{argmax}_{\underline{h}} \left(\prod_{j=1}^N P(v_j | h_j) P(h_1) \prod_{i=1}^{N-1} P(h_{i+1} | h_i) \right)$$

中文分词的基本算法：HMM



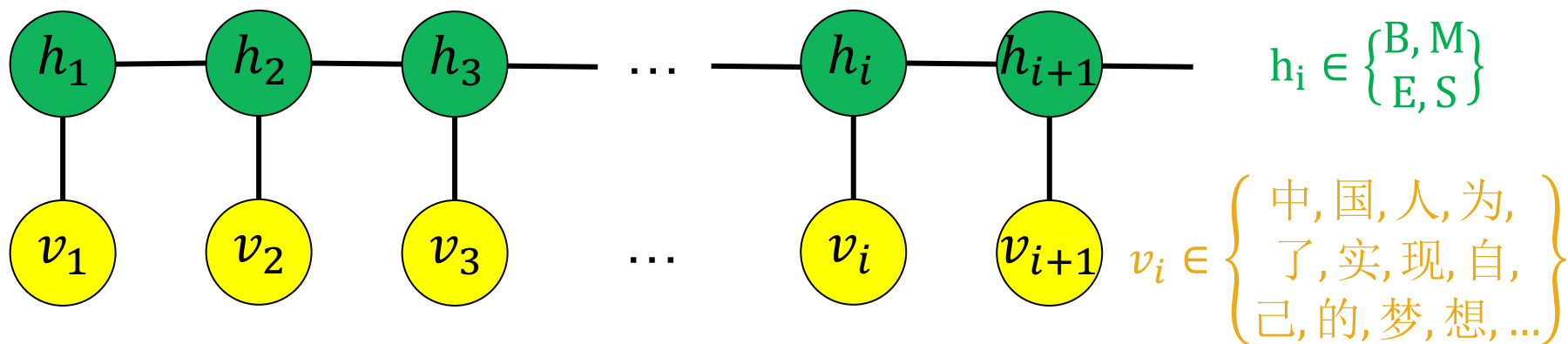
$$\operatorname{argmax}_{\underline{h}} \underline{P(\underline{h} | \underline{v})} = \operatorname{argmax}_{\underline{h}} \cancel{P(\underline{h} | \underline{v})}$$

$$= \operatorname{argmax}_{\underline{h}} \cancel{(P(\underline{h}) \cdot P(\underline{h}))}$$

$$= \operatorname{argmax}_{\underline{h}} \cancel{\left(\prod_{j=1}^N P(v_j | h_j) P(h_j) \prod_{i=1}^{N-1} P(h_{i+1} | h_i) \right)}$$

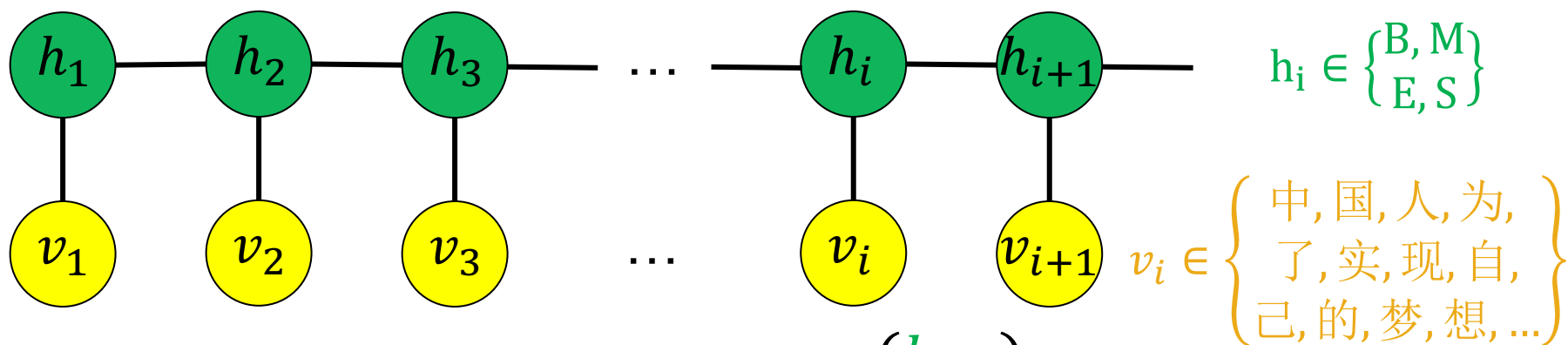
不通过贝叶斯公式，
而是直接对条件概率
简单直接地进行拟合

中文分词的基本算法：CRF概念



$$\operatorname{argmax}_{\underset{h}{\text{green}}} P(\underset{h}{\text{green}} | \underset{v}{\text{orange}}) = \operatorname{argmax}_{\underset{h}{\text{green}}}$$

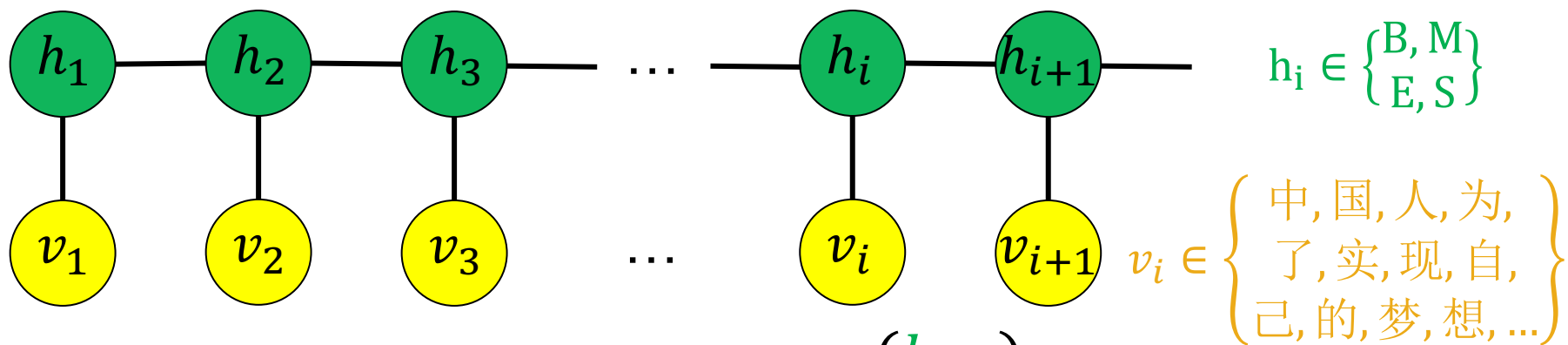
中文分词的基本算法：CRF概念



$$\operatorname{argmax}_{\underset{h}{h}} P(\underset{h}{h} | \underset{v}{v}) = \operatorname{argmax}_{\underset{h}{h}} \frac{\operatorname{score}(\underset{h}{h}, \underset{v}{v})}{\sum_{\underset{h}{h}} \operatorname{score}(\underset{h}{h}, \underset{v}{v})}$$

$$\operatorname{score}(\underset{h}{h}, \underset{v}{v}) = F_k(\underset{h}{h}, \underset{v}{v})$$

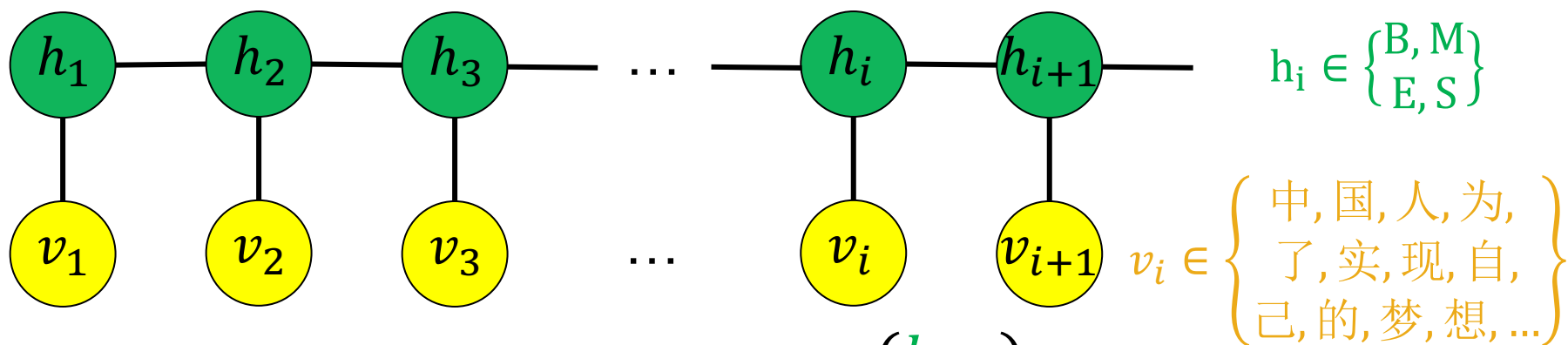
中文分词的基本算法：CRF概念



$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h}|\mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \frac{\operatorname{score}(\mathbf{h}, \mathbf{v})}{\sum_{\mathbf{h}} \operatorname{score}(\mathbf{h}, \mathbf{v})}$$

$$\operatorname{score}(\mathbf{h}, \mathbf{v}) = \sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v})$$

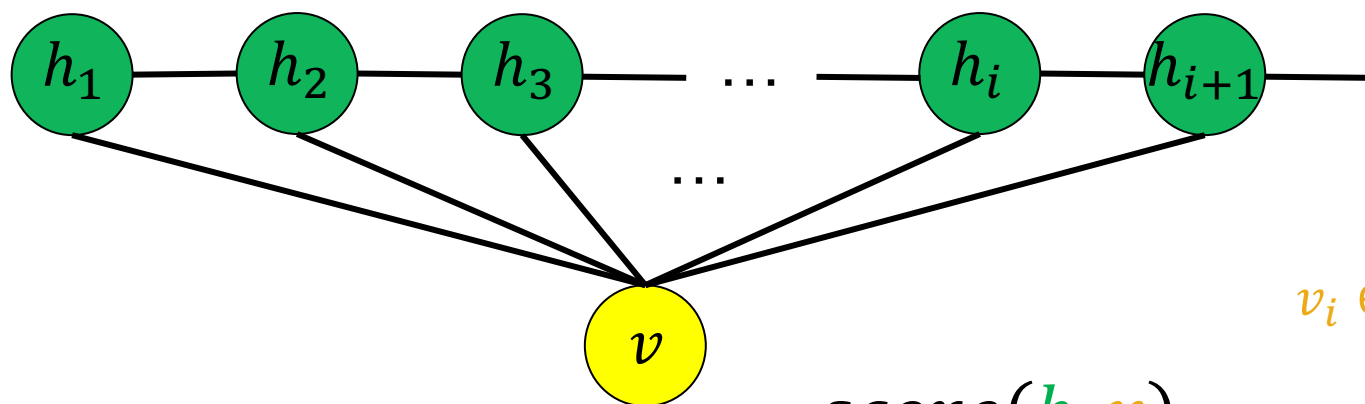
中文分词的基本算法：CRF概念



$$\operatorname{argmax}_{\underline{h}} P(\underline{h}|\underline{v}) = \operatorname{argmax}_{\underline{h}} \frac{\operatorname{score}(\underline{h}, \underline{v})}{\sum_{\underline{h}} \operatorname{score}(\underline{h}, \underline{v})}$$

$$\operatorname{score}(\underline{h}, \underline{v}) = \exp \left(\sum_{k=1}^K w_k F_k(\underline{h}, \underline{v}) \right)$$

中文分词的基本算法：CRF概念



$h_i \in \begin{Bmatrix} B, M \\ E, S \end{Bmatrix}$

$v_i \in \left\{ \begin{array}{l} \text{中, 国, 人, 为,} \\ \text{了, 实, 现, 自,} \\ \text{己, 的, 梦, 想, ...} \end{array} \right\}$

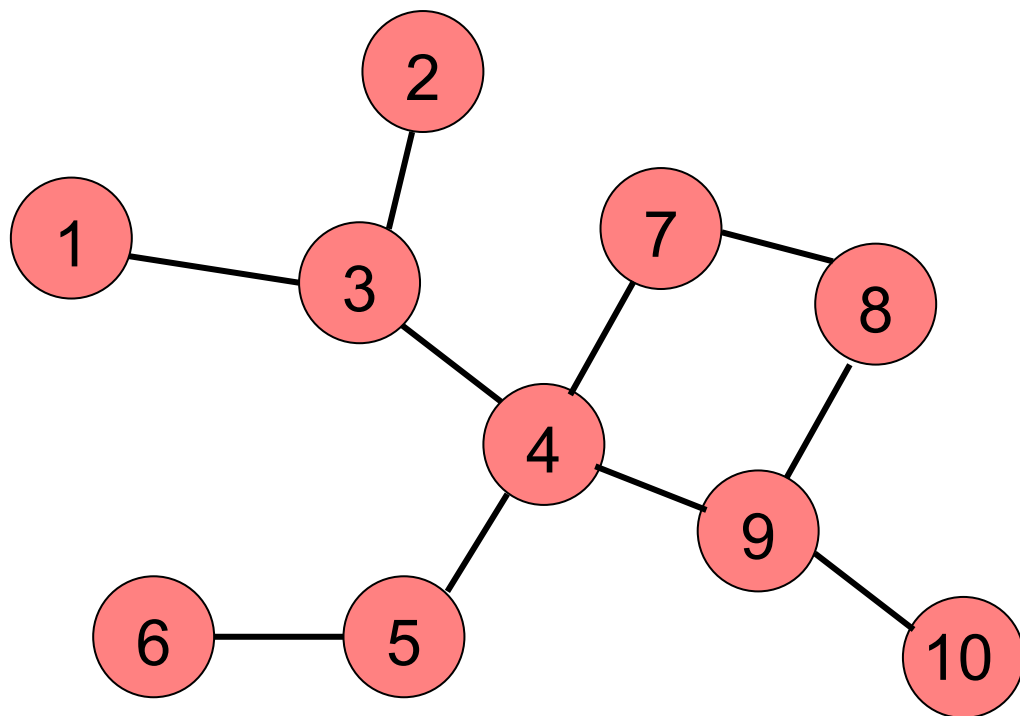
$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h}|\mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \frac{\operatorname{score}(\mathbf{h}, \mathbf{v})}{\sum_{\mathbf{h}} \operatorname{score}(\mathbf{h}, \mathbf{v})} \quad (\operatorname{score}(\mathbf{h}, \mathbf{v}) \geq 0)$$

$$\operatorname{score}(\mathbf{h}, \mathbf{v}) = \exp \left(\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right)$$

$$\begin{aligned} F_k(\mathbf{h}, \mathbf{v}) &= F_k(h_1, h_2, \dots, h_i, \dots, h_N, v_1, v_2, \dots, v_i, \dots, v_N) \\ &= \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \end{aligned}$$

概率图模型

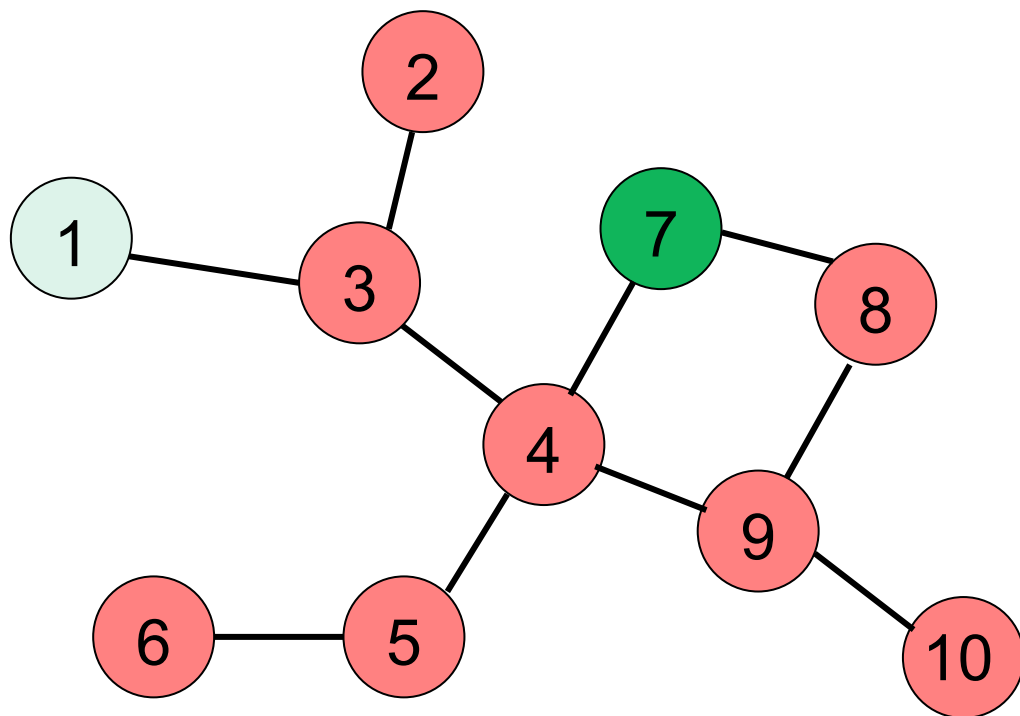
- ▶ 概率图模型是由图表示的联合概率分布 $P(Y)$
- ▶ 结点 $v \in V$ 表示一个随机变量 Y_v
- ▶ 边 $e \in E$ 表示随机变量之间的依赖关系



概率图模型：成对马尔科夫性

- ▶ v 和 u 是图中任意两个不直接相连的结点
- ▶ O 是图中其它所有结点的集合

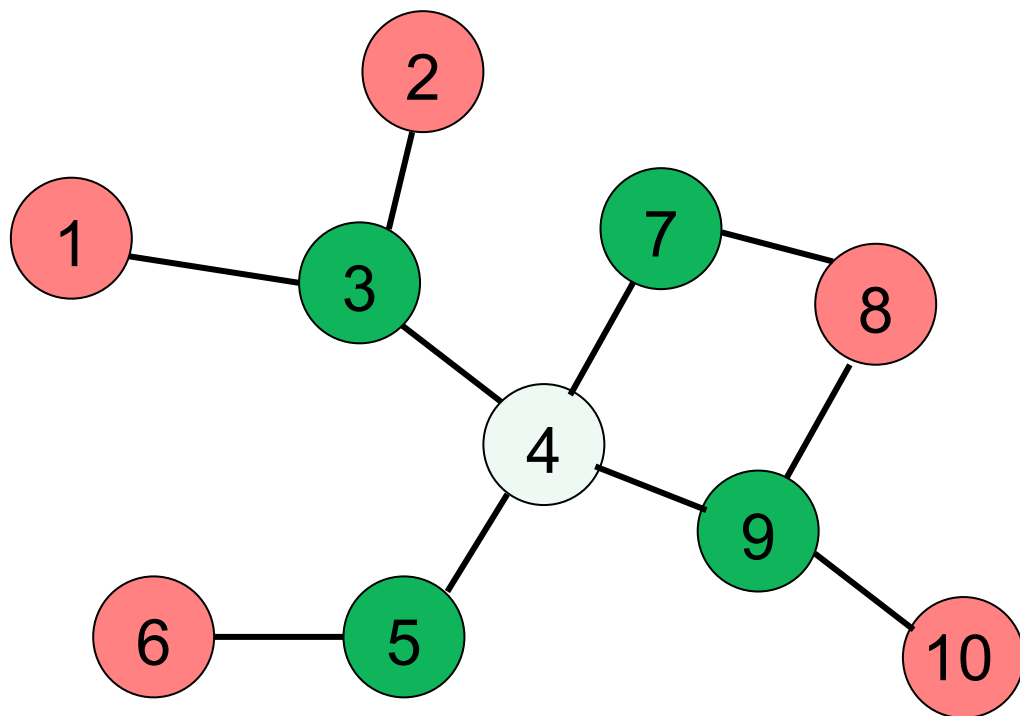
$$P(Y_v, Y_u | Y_O) = P(Y_v | Y_O) P(Y_u | Y_O)$$



概率图模型：局部马尔科夫性

- ▶ v 是图中任意一个结点， W 是与 v 相连的所有结点
- ▶ O 是图中除 v 和 W 所有结点的集合

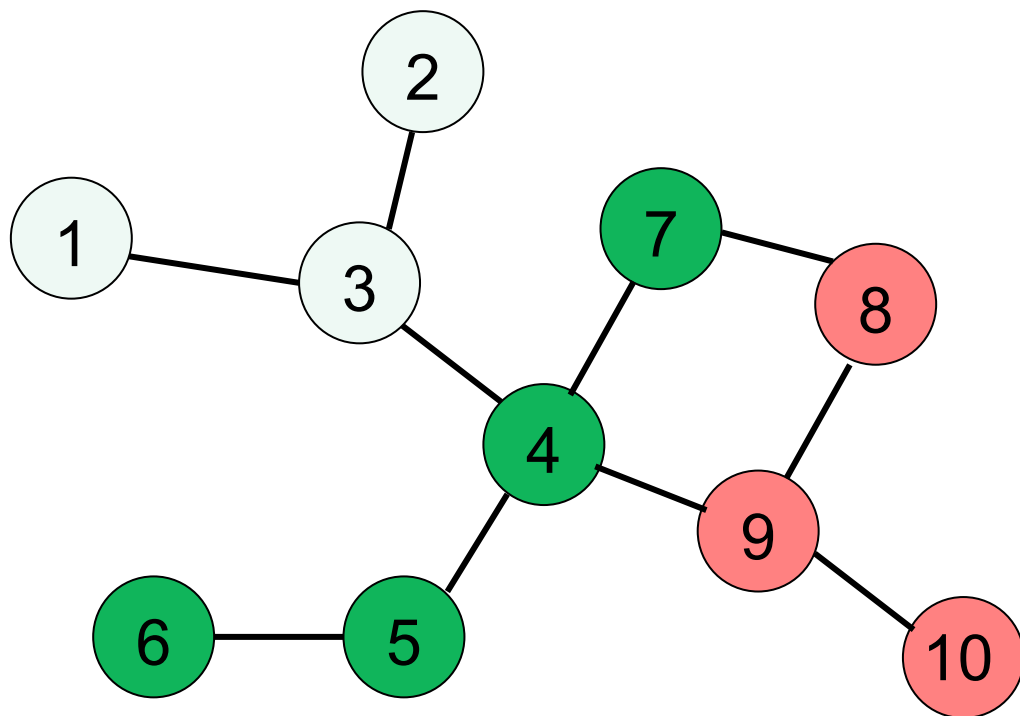
$$P(Y_v, Y_O | Y_W) = P(Y_v | Y_W) P(Y_O | Y_W)$$



概率图模型：全局马尔科夫性

- ▶ 结点集合 A 和 B 是被结点集合 C 隔开的两个结点集合

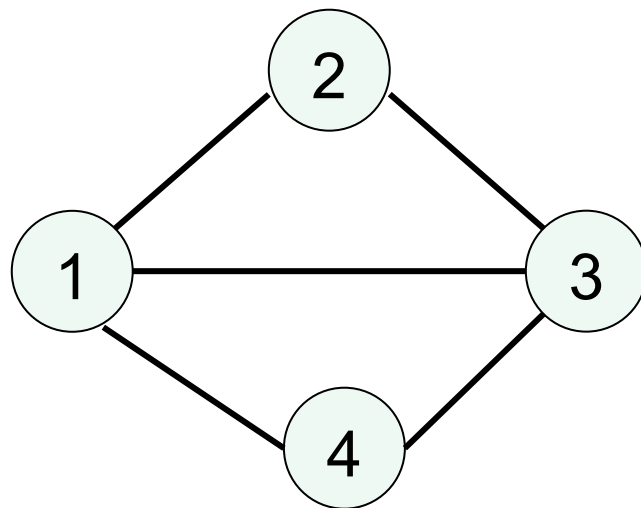
$$P(Y_A, Y_B | Y_C) = P(Y_A | Y_C) P(Y_B | Y_C)$$



定义：设有联合概率分布 $P(Y)$ ，由无向图 $G = (V, E)$ 表示，结点表示随机变量，边表示随机变量之间的依赖关系。如果联合概率分布 $P(Y)$ 满足成对、局部或全局马尔科夫性，就称此联合概率分布为概率（无向）图模型，或马尔科夫随机场。

概率图模型的因子分解

定义：无向图 $G = (V, E)$ 中任意两个结点之间均有边连接的结点子集称为一个**团** (clique)。若 C 是图中的一个团，并且不能再加进任何一个的结点使其成为一个更大的团，则称此团为**最大团** (maximal clique)。



定理： 概率图模型的联合概率分布 $P(Y)$ 可表示成如下形式

$$P(Y) = \frac{1}{Z} \prod_C \Psi_C(Y_C)$$

其中， C 是图中的最大团， $\Psi_C(Y_C)$ 是 C 上定义的严格正函数，通常定义为指数函数 $\Psi_C(Y_C) = \exp\{E(Y_C)\}$

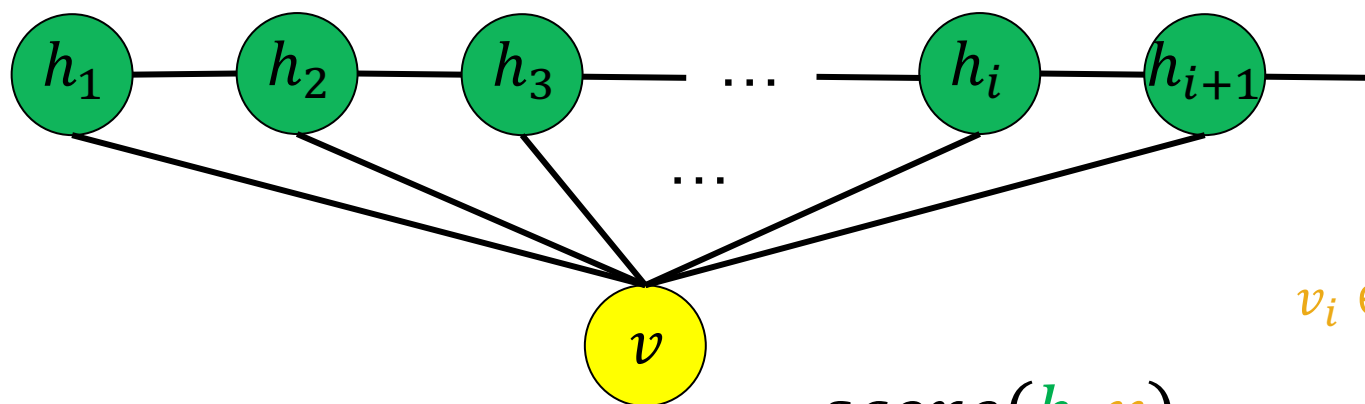
条件随机场 (CRF) : 设 X 与 Y 是随机变量, $P(Y|X)$ 是在给定 X 的条件下 Y 的条件概率分布, 若随机变量 Y 构成一个由图表示的马尔科夫随机场, 则称 $P(Y|X)$ 为条件概率随机场。

线性条件随机场 (linear CRF) : 设 $X = (X_1, X_2, \dots, X_n)$ 与 $Y = (Y_1, Y_2, \dots, Y_n)$ 是均为线性链表示的随机变量序列, 若 $P(Y|X)$ 构成条件随机场, 即满足:

$$P(Y_i | X, Y_{\sim i}) = P(Y_i | X, Y_{i-1}, Y_{i+1})$$

则称 $P(Y|X)$ 为线性条件随机场。

中文分词的基本算法：CRF概念



$h_i \in \begin{Bmatrix} B, M \\ E, S \end{Bmatrix}$

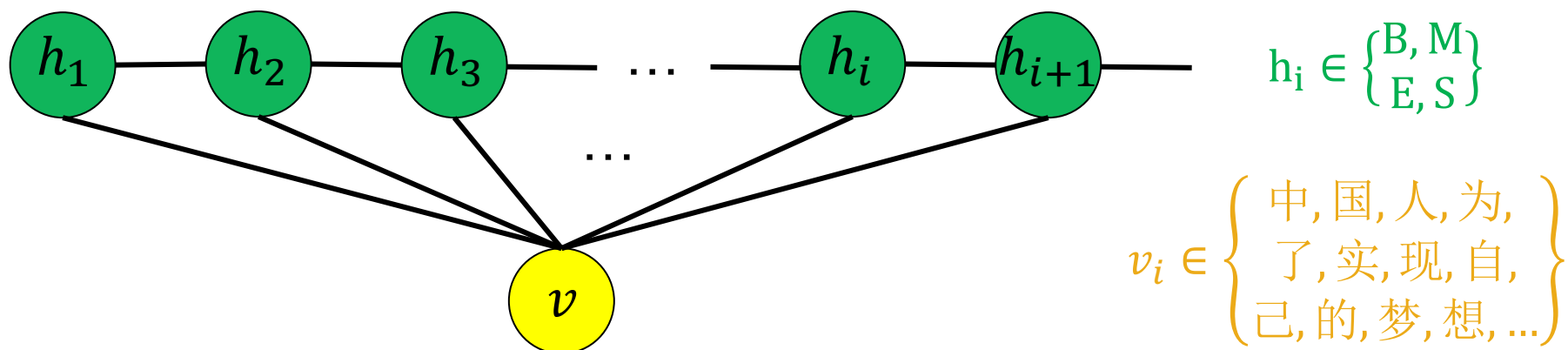
$v_i \in \left\{ \begin{array}{l} \text{中, 国, 人, 为,} \\ \text{了, 实, 现, 自,} \\ \text{己, 的, 梦, 想, ...} \end{array} \right\}$

$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h}|\mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \frac{\operatorname{score}(\mathbf{h}, \mathbf{v})}{\sum_{\mathbf{h}} \operatorname{score}(\mathbf{h}, \mathbf{v})} \quad (\operatorname{score}(\mathbf{h}, \mathbf{v}) \geq 0)$$

$$\operatorname{score}(\mathbf{h}, \mathbf{v}) = \exp \left(\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right)$$

$$\begin{aligned} F_k(\mathbf{h}, \mathbf{v}) &= F_k(h_1, h_2, \dots, h_i, \dots, h_N, v_1, v_2, \dots, v_i, \dots, v_N) \\ &= \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \end{aligned}$$

中文分词的基本算法：CRF概念

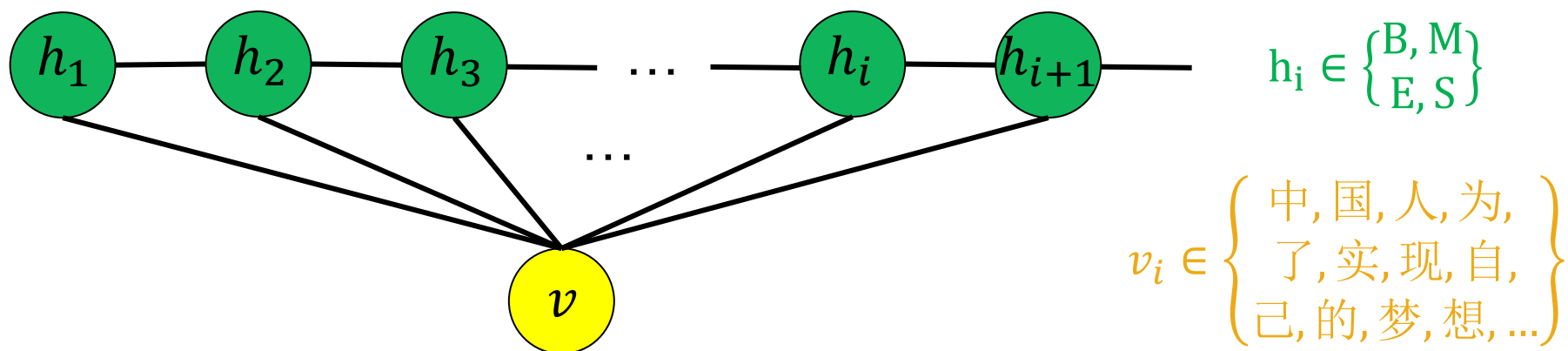


$$P(h|v) = \frac{\text{score}(h, v)}{\sum_h \text{score}(h, v)} \quad (\text{s.t.} \quad \text{score}(h, v) \geq 0)$$

$$\text{score}(h, v) = \exp \left(\sum_{k=1}^K w_k F_k(h, v) \right)$$

$$\begin{aligned} F_k(h, v) &= F_k(h_1, h_2, \dots, h_i, \dots, h_N, v_1, v_2, \dots, v_i, \dots, v_N) \\ &= \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \end{aligned}$$

中文分词的基本算法：CRF概念



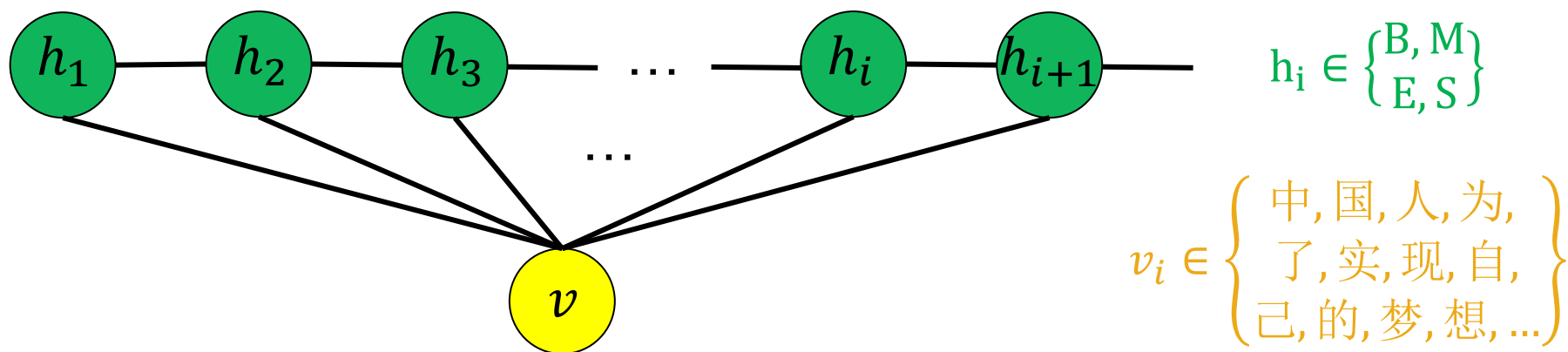
$$P(h|v) = \frac{\text{score}(h, v)}{\sum_h \text{score}(h, v)} \quad (\text{s.t.} \quad \text{score}(h, v) \geq 0)$$

$$\begin{aligned} &= \frac{1}{Z(v)} \exp \left(\sum_{k=1}^K w_k \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \right) \\ &= \frac{1}{Z(v)} \prod_{i=2}^N \Psi_i(h_i, h_{i-1}, v) \end{aligned}$$

$$P(Y) = \frac{1}{Z} \prod_C \Psi_C(Y_C)$$

$$\text{其中 } \Psi_i(h_i, h_{i-1}, v) = \exp \left(\sum_{k=1}^K w_k f_k(h_i, h_{i-1}, v, i) \right)$$

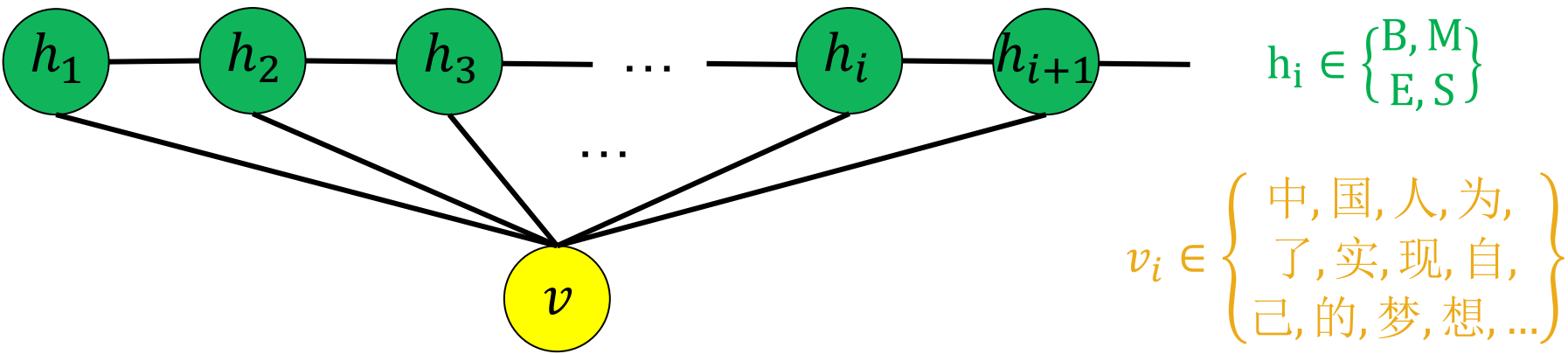
中文分词的基本算法：CRF概念



$f_k(h_i, h_{i-1}, v, i)$ 的选取:

- $f_k(h_i, h_{i-1}, v, i)$ 通常是值域为 $\{0, 1\}$ 的二值函数，在满足特征规定的条件后值为一，否则为零。
- $f_k(h_i, h_{i-1}, v, i)$ 的具体定义可以非常灵活。可以只与输入序列 v 有关，也可以与 h_i, h_{i-1} 和 v 的对应关系有关，甚至可以与feature所在的具体的位置 i 有关。

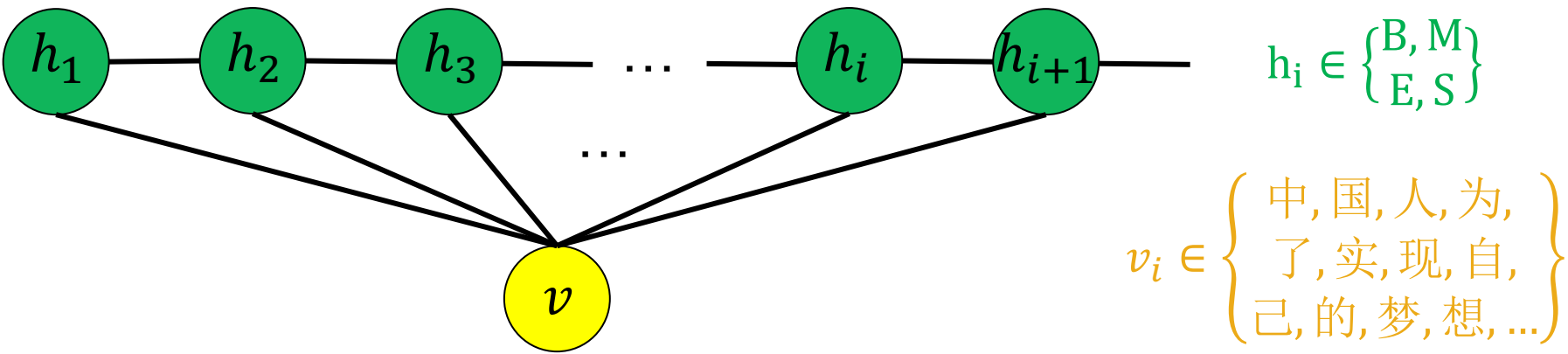
中文分词的基本算法：CRF概念



$f_k(h_i, h_{i-1}, v, i)$ 的选取:

类型	特征	特征激活条件
一元特征	v_{i-1}	上一字符为某特定字符
	v_i	当前字符为某特定字符
	v_{i+1}	下一字符为某特定字符
二元特征	$\overline{v_{i-1}v_i}$	当前字符和上一字符为某特定串
	$\overline{v_i v_{i+1}}$	当前字符和下一字符为某特征串
跳跃特征	$\overline{v_{i-1} \blacksquare v_{i+1}}$	上一字符和下一字符为某特定串
三元特征	$\overline{v_{i-1}v_i v_{i+1}}$	上一字符、当前字符、下一字符为某特定串
.....		

中文分词的基本算法：CRF概念



$f_k(h_i, h_{i-1}, v, i)$ 的选取:

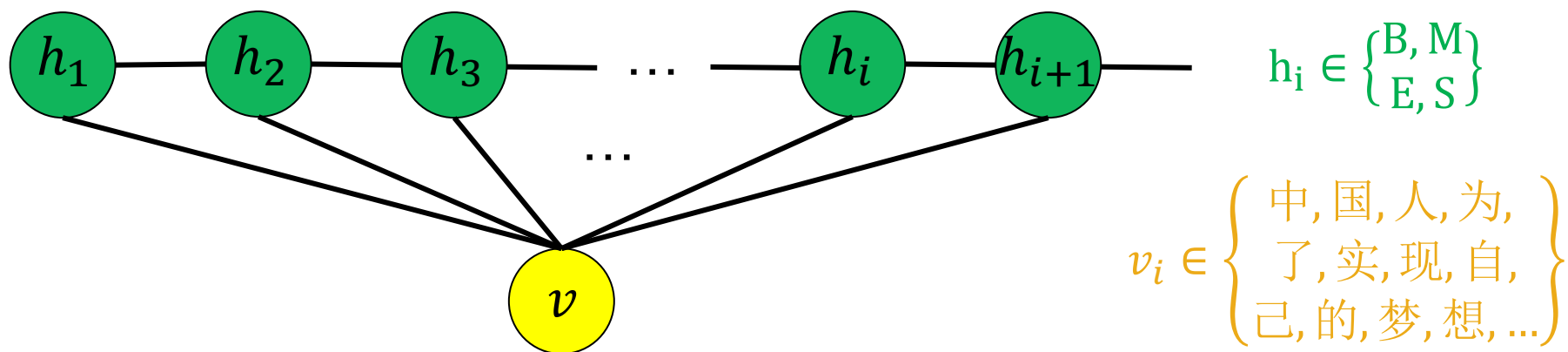
类型	特征	特征激活条件
状态转移	$\overline{h_{i-1}}h_i$	上一字符的标记 h_{i-1} 转移到当前字符的标记 h_i
一元状态特征	h_{i-1}	上一字符为某特定标记
	h_i	当前字符为某特定标记
混合特征	$\overline{v_{i-1}v_i} \cap h_i \cap (i = 5)$	上一字符和当前字符为某特定串, 且 当前字符的标记为 h_i , 且 当前所在位置为5

.....

.....

.....

中文分词的基本算法：CRF概念



$f_k(h_i, h_{i-1}, v, i)$ 的选取:

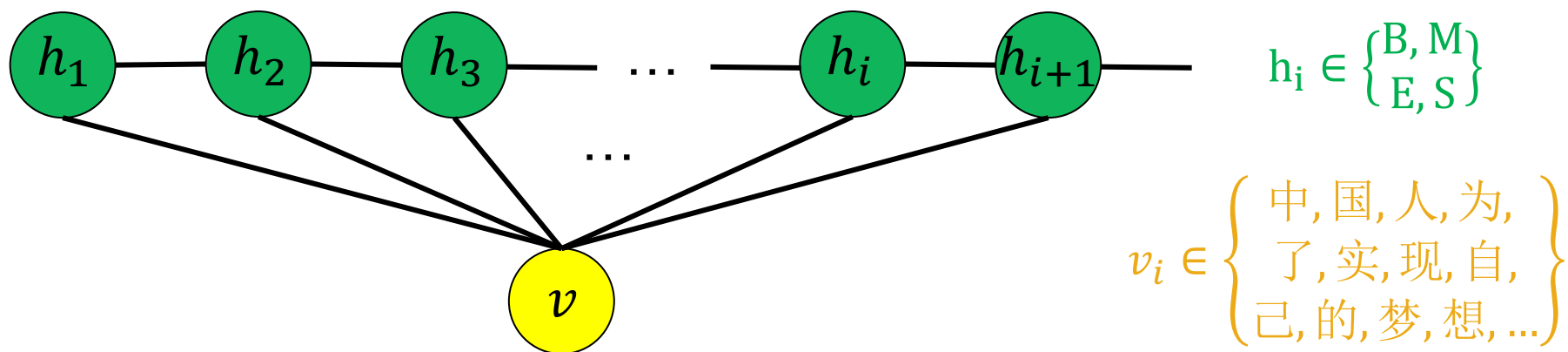
- 对一个具有实用价值的CRF分词器而言，其使用的特征 (feature) 数目 k 在

$10^5 \sim 10^6$

量级。

- 因而，这些特征多是由特征模板 (feature template) 生成的。

中文分词的基本算法：CRF概念



$f_k(h_i, h_{i-1}, v, i)$ 的选取:

- 特征模板 (feature template) :

给定某特征模板, 比如

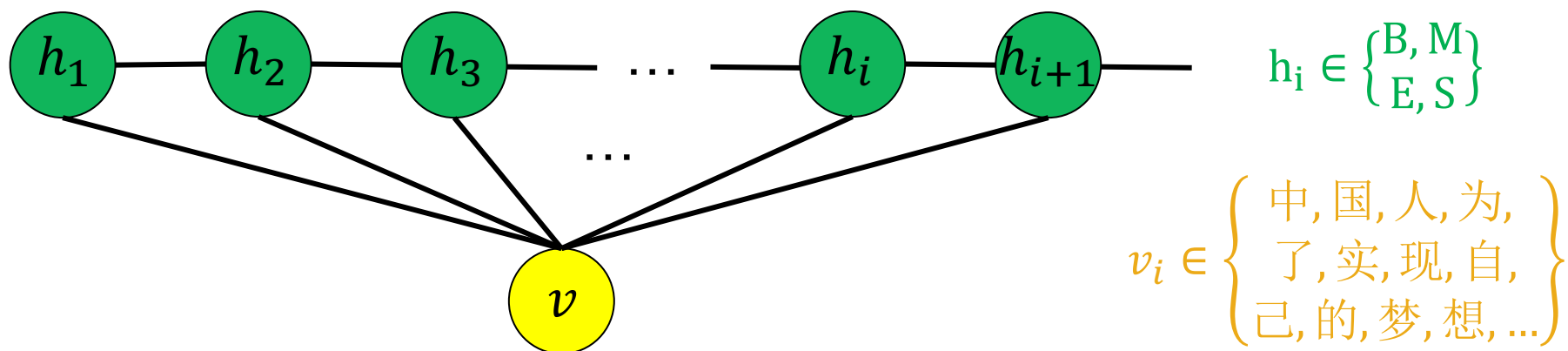
$$\overline{v_{i-1}v_i} \cap h_i \cap (i = 5)$$

算法将依照一定的规则在数据集中遍历, 找到 N 个符合该形式的具体特征。
(比如最常出现的 N 个 v_{i-1} , v_i , h_i 和 i 的组合)

- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

中文分词的基本算法：CRF概念

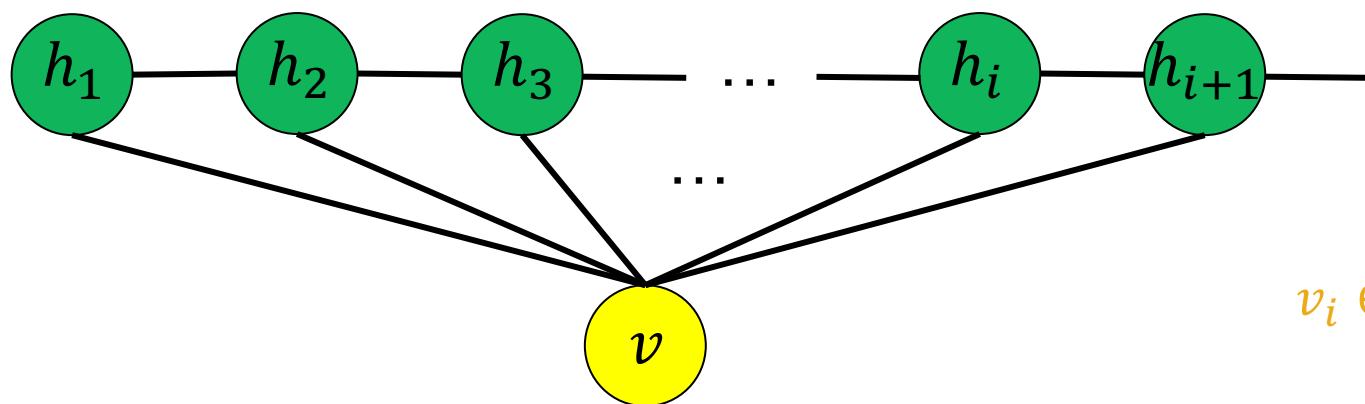


$$P(h|v) = \frac{\text{score}(h, v)}{\sum_h \text{score}(h, v)} = \frac{1}{Z(v)} \prod_{i=2}^N \Psi_i(h_i, h_{i-1}, v)$$

其中 $\Psi_i(h_i, h_{i-1}, v) = \exp(\sum_{k=1}^K w_k f_k(h_i, h_{i-1}, v, i))$

$$Z(v) = \sum_h \prod_{i=2}^N \Psi_i(h_i, h_{i-1}, v)$$

条件随机场：前向算法与后向算法



$h_i \in \begin{Bmatrix} \text{B, M} \\ \text{E, S} \end{Bmatrix}$

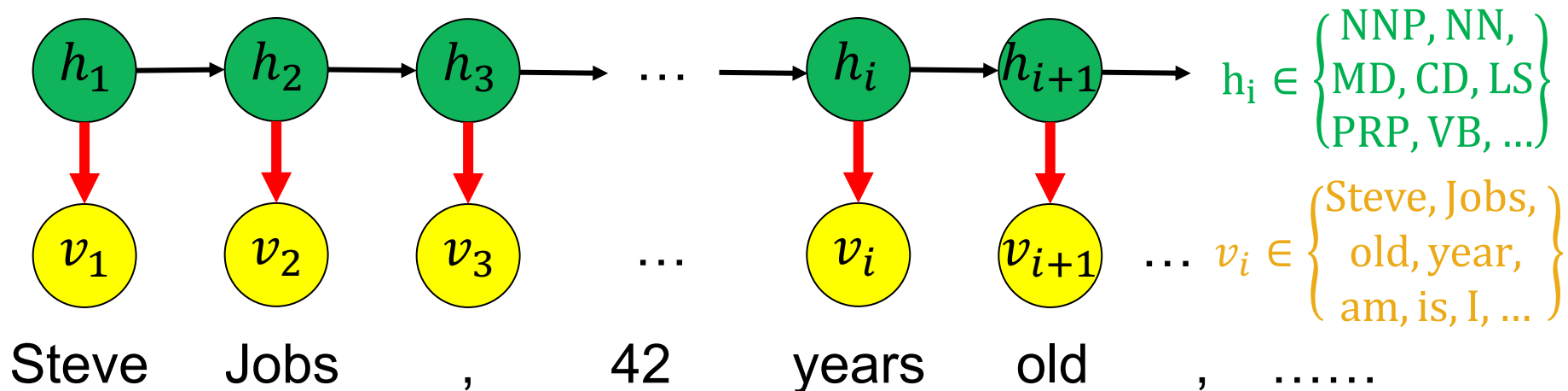
$v_i \in \left\{ \begin{array}{l} \text{中, 国, 人, 为,} \\ \text{了, 实, 现, 自,} \\ \text{己, 的, 梦, 想, ...} \end{array} \right\}$

$$Z(\mathbf{v}) = \sum_{h_1, h_2, \dots, h_N} \prod_{i=2}^N \Psi_i(h_i, h_{i-1}, \mathbf{v})$$

$$= \sum_{h_N} \left\{ \exp \left(\sum_{k=1}^K w_k f_k(h_N, h_{N-1}, \mathbf{v}, i) \right) \cdots \sum_{h_2} \left[\exp \left(\sum_{k=1}^K w_k f_k(h_3, h_2, \mathbf{v}, i) \right) \sum_{h_1} \exp \left(\sum_{k=1}^K w_k f_k(h_2, h_1, \mathbf{v}, 1) \right) \right] \right\}$$

$$= \sum_{h_N} \left\{ \exp \left(\sum_{k=1}^K w_k f_k(h_N, h_{N-1}, \mathbf{v}, i) \right) \cdots \sum_{h_2} \left[\exp \left(\sum_{k=1}^K w_k f_k(h_3, h_2, \mathbf{v}, i) \right) \sum_{h_1} \exp \left(\sum_{k=1}^K w_k f_k(h_2, h_1, \mathbf{v}, 1) \right) \right] \right\}$$

回顾：隐马尔科夫模型前向算法



$$P(v) = P(v_1, v_2, \dots, v_N)$$

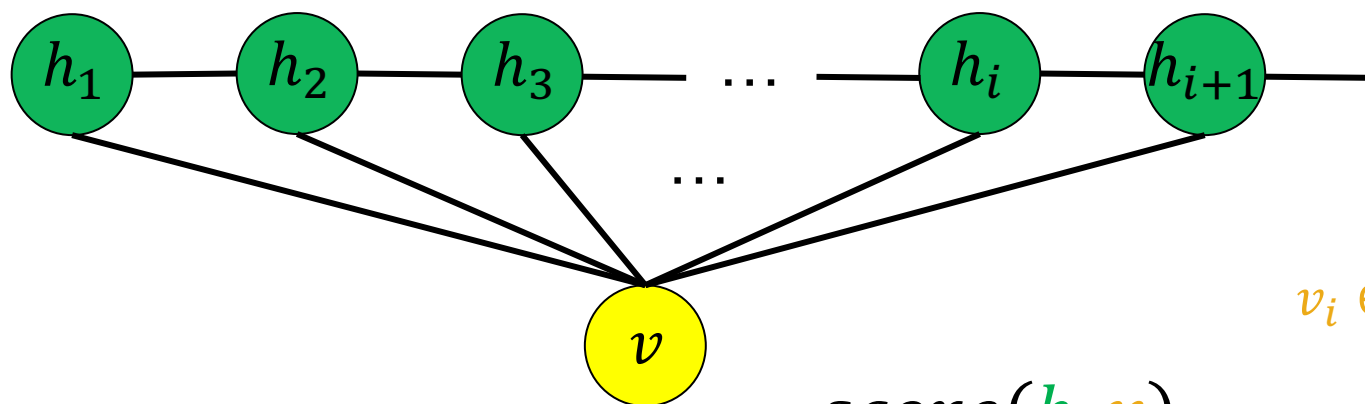
$$\alpha_{t+1}(j) = \sum_{i=1}^S \alpha_t(i) P(h_{t+1} = j | h_t = i) P(v_{t+1} | h_{t+1} = j)$$

$$\begin{aligned}
 &= \sum_{h_N} P(v_1, v_2, \dots, v_N, h_N) \\
 &= \sum_{h_N} \alpha_N(j) \\
 &= \sum_{h_N} \sum_{h_{N-1}} \alpha_{N-1}(i) P(h_N | h_{N-1}) P(v_N | h_N) \\
 &= \sum_{h_N} \sum_{h_{N-1}} \alpha_{N-1}(i) P(h_N | h_{N-1}) P(v_N | h_N) \\
 &= \sum_{h_N} \sum_{h_{N-1}} P(h_N | h_{N-1}) P(v_N | h_N) \sum_{h_{N-2}} \alpha_{N-2}(i) P(h_{N-1} | h_{N-2}) P(v_{N-1} | h_{N-1}) \\
 &= \sum_{h_N} \sum_{h_{N-1}} \left\{ P(h_N | h_{N-1}) P(v_N | h_N) \dots \sum_{h_{N-2}} \dots \sum_{h_2} [\alpha_1(i) P(h_2 | h_1) P(v_2 | h_2)] \right\}
 \end{aligned}$$



- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

条件随机场：Viterbi算法



$h_i \in \begin{Bmatrix} B, M \\ E, S \end{Bmatrix}$

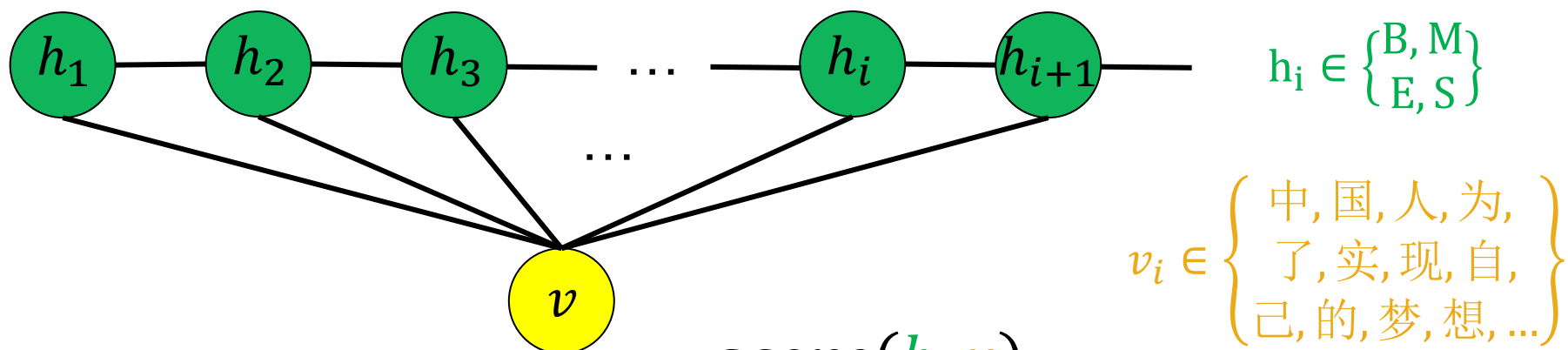
$v_i \in \left\{ \begin{array}{l} \text{中, 国, 人, 为,} \\ \text{了, 实, 现, 自,} \\ \text{己, 的, 梦, 想, ...} \end{array} \right\}$

$$\operatorname{argmax}_{\underset{h}{h}} P(\underset{h}{h} | \underset{v}{v}) = \operatorname{argmax}_{\underset{h}{h}} \frac{\operatorname{score}(\underset{h}{h}, \underset{v}{v})}{\sum_{\underset{h}{h}} \operatorname{score}(\underset{h}{h}, \underset{v}{v})} \quad (\operatorname{score}(\underset{h}{h}, \underset{v}{v}) \geq 0)$$

$$= \operatorname{argmax}_{\underset{h}{h}} \frac{\exp(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v}))}{\sum_{\underset{h}{h}} \exp(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v}))}$$

$$= \operatorname{argmax}_{\underset{h}{h}} \frac{\exp(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v}))}{\cancel{Z(\underset{v}{v})}}$$

条件随机场： Viterbi算法

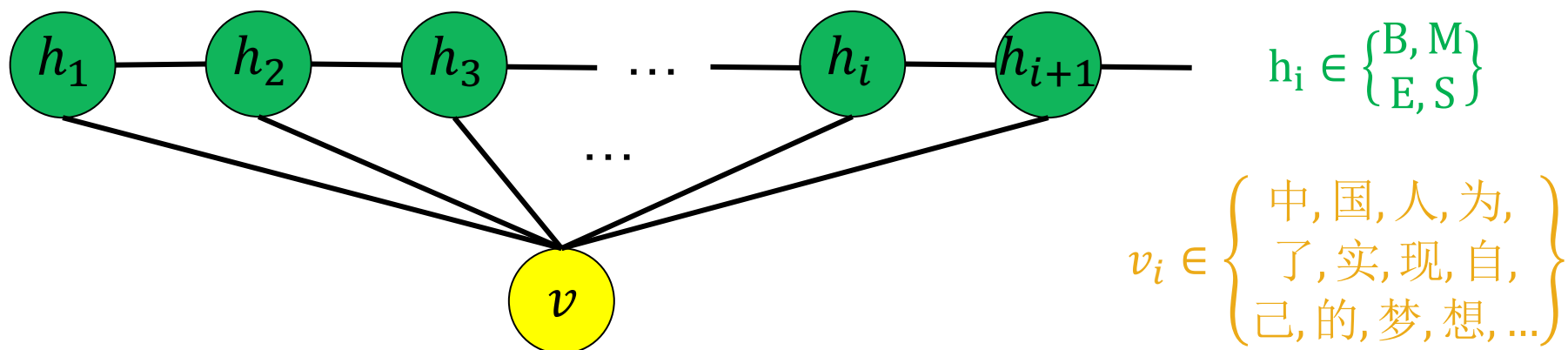


$$\operatorname{argmax}_{\underset{h}{h}} P(\underset{h}{h} | \underset{v}{v}) = \operatorname{argmax}_{\underset{h}{h}} \frac{\operatorname{score}(\underset{h}{h}, \underset{v}{v})}{\sum_{\underset{h}{h}} \operatorname{score}(\underset{h}{h}, \underset{v}{v})} \quad (\operatorname{score}(\underset{h}{h}, \underset{v}{v}) \geq 0)$$

$$= \operatorname{argmax}_{\underset{h}{h}} \frac{\exp\left(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v})\right)}{\sum_{\underset{h}{h}} \exp\left(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v})\right)}$$

$$= \operatorname{argmax}_{\underset{h}{h}} \exp\left(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v})\right)$$

条件随机场： Viterbi算法

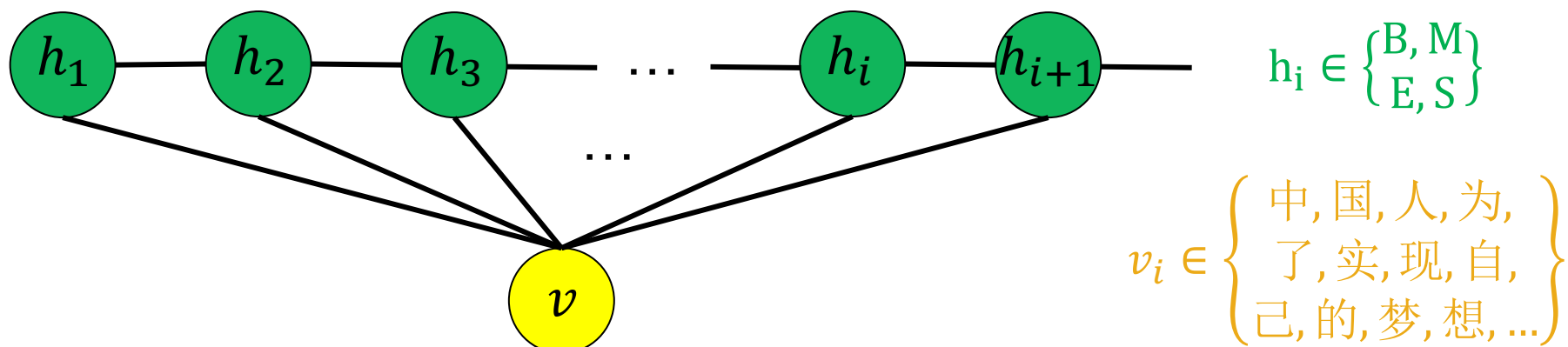


$$\operatorname{argmax}_{\underline{h}} P(\underline{h} | \underline{v}) = \operatorname{argmax}_{\underline{h}} \left[\exp \left(\sum_{k=1}^K w_k F_k(\underline{h}, \underline{v}) \right) \right]$$

$$= \operatorname{argmax}_{\underline{h}} \left[\sum_{k=1}^K w_k F_k(\underline{h}, \underline{v}) \right]$$

$$= \operatorname{argmax}_{\underline{h}} \left[\sum_{k=1}^K w_k \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \right]$$

条件随机场：Viterbi算法

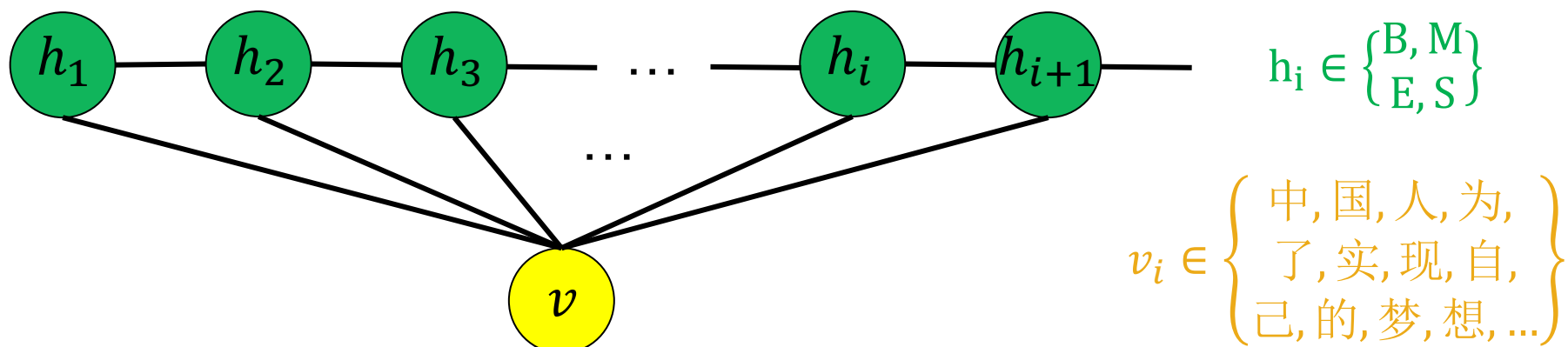


$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h} | \mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \left[\exp \left(\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right) \right]$$

$$= \operatorname{argmax}_{\mathbf{h}} \left[\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right] = \operatorname{argmax}_{\mathbf{h}} \left[\sum_{k=1}^K w_k \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \right]$$

$$= \operatorname{argmax}_{\mathbf{h}} \left[\sum_{k=1}^K \sum_{i=2}^N w_k f_k(h_i, h_{i-1}, v, i) \right]$$

条件随机场：Viterbi算法

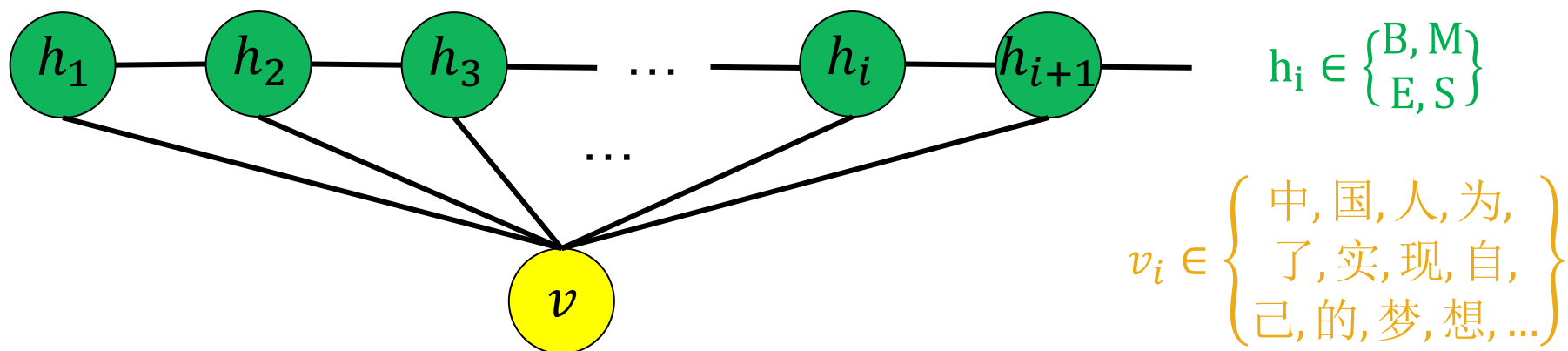


$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h} | \mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \left[\exp \left(\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right) \right]$$

$$= \operatorname{argmax}_{\mathbf{h}} \left[\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right] = \operatorname{argmax}_{\mathbf{h}} \left[\sum_{k=1}^K w_k \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \right]$$

$$= \operatorname{argmax}_{\mathbf{h}} \left[\sum_{i=2}^N \sum_{k=1}^K w_k f_k(h_i, h_{i-1}, v, i) \right]$$

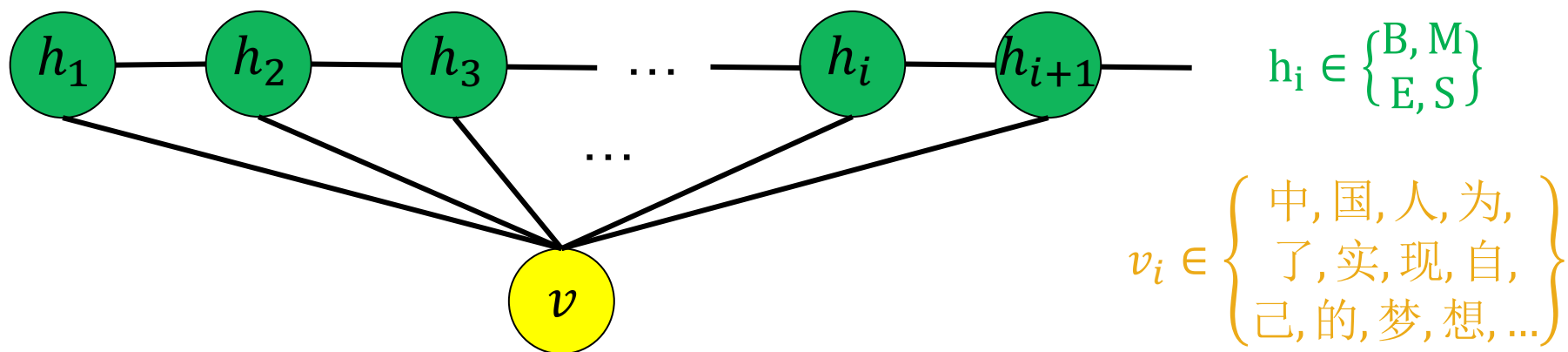
条件随机场：Viterbi算法



$$\operatorname{argmax}_{\underset{h}{h}} P(\underset{h}{h} | \underset{v}{v}) = \operatorname{argmax}_{\underset{h}{h}} \left[\exp \left(\sum_{k=1}^K w_k F_k(\underset{h}{h}, \underset{v}{v}) \right) \right]$$

$$= \operatorname{argmax}_{\underset{h}{h}} \left[\sum_{i=2}^N \sum_{k=1}^K w_k f_k(\underset{h}{h}_i, \underset{h}{h}_{i-1}, \underset{v}{v}, i) \right]$$

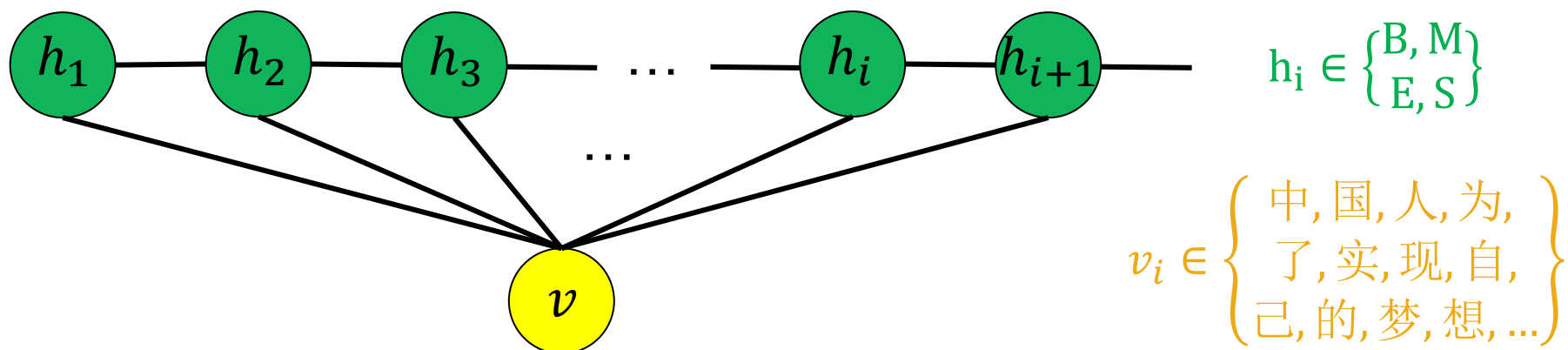
条件随机场：Viterbi算法



$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h} | \mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \left[\exp \left(\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right) \right]$$

$$= \operatorname{argmax}_{h_1, h_2, \dots, h_i, \dots, h_N} \left[\sum_{k=1}^K w_k f_k(h_2, h_1, v, 2) + \sum_{k=1}^K w_k f_k(h_3, h_2, v, 3) + \dots \right. \\ \left. + \sum_{k=1}^K w_k f_k(h_N, h_{N-1}, v, N) \right]$$

条件随机场：Viterbi算法



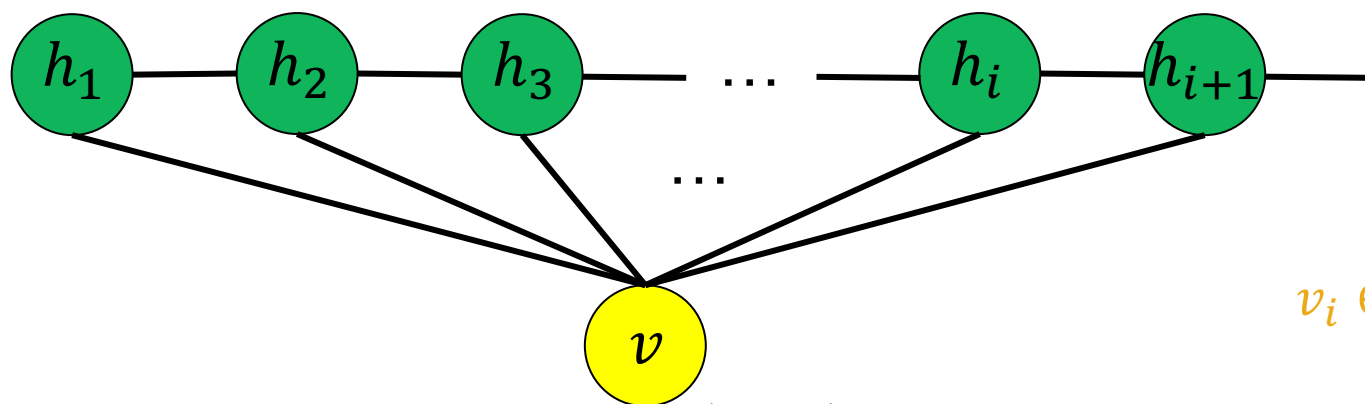
$$\operatorname{argmax}_{\mathbf{h}} P(\mathbf{h} | \mathbf{v}) = \operatorname{argmax}_{\mathbf{h}} \left[\exp \left(\sum_{k=1}^K w_k F_k(\mathbf{h}, \mathbf{v}) \right) \right]$$

$$= \operatorname{argmax}_{h_N} \left\{ \operatorname{argmax}_{h_{N-1}} \left\{ \sum_{k=1}^K w_k f_k(h_N, h_{N-1}, \mathbf{v}, N) + \cdots + \operatorname{argmax}_{h_2} \left[\sum_{k=1}^K w_k f_k(h_3, h_2, \mathbf{v}, 3) + \operatorname{argmax}_{h_1} \sum_{k=1}^K w_k f_k(h_2, h_1, \mathbf{v}, 2) \right] \right\} \right\}$$

计算过程与HMM中的Viterbi算法大同小异。

- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

条件随机场



$h_i \in \begin{Bmatrix} B, M \\ E, S \end{Bmatrix}$

$v_i \in \begin{Bmatrix} \text{中, 国, 人, 为,} \\ \text{了, 实, 现, 自,} \\ \text{己, 的, 梦, 想, ...} \end{Bmatrix}$

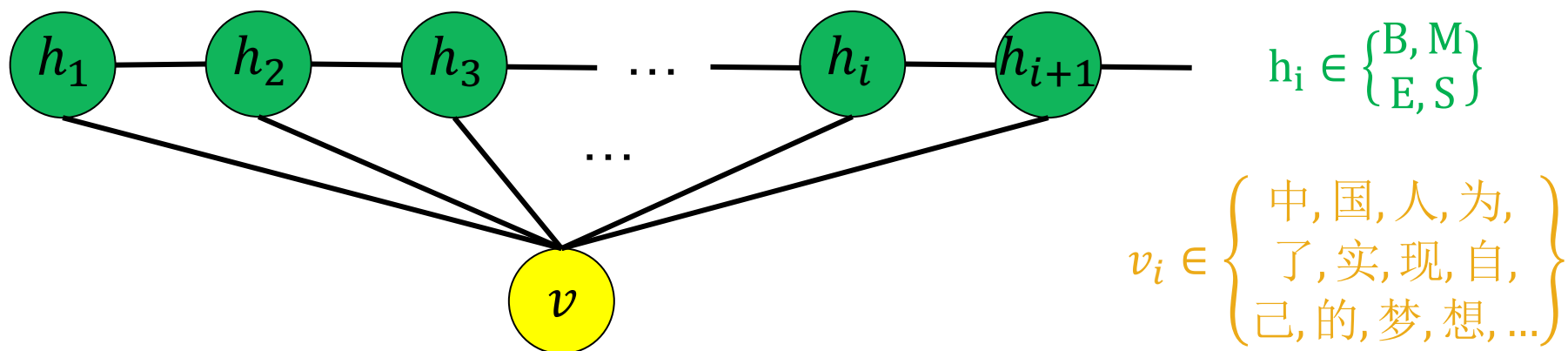
$$P(h|v) = \frac{\text{score}(h, v)}{\sum_h \text{score}(h, v)}$$

($\text{score}(h, v) \geq 0$)

$$\sum_h \text{score}(h, v) = \sum_h \exp \left(\sum_{k=1}^K w_k F_k(h, v) \right) = Z(v)$$

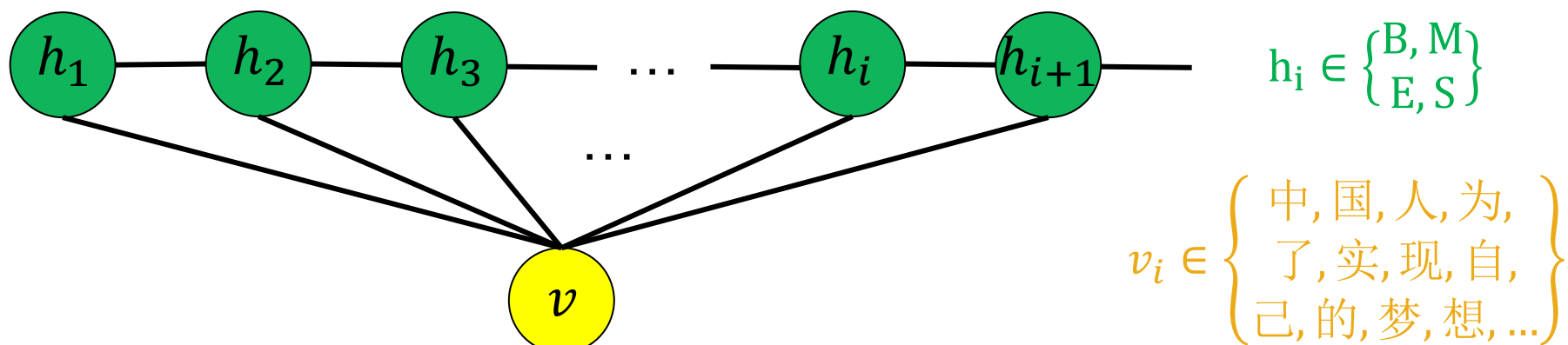
$$Z(v) = \sum_h \exp \left(\sum_{k=1}^K w_k \sum_{i=2}^N f_k(h_i, h_{i-1}, v, i) \right)$$

条件随机场：训练



$$\begin{aligned} \max_w L(w) &= \max_w \sum_{s=1}^M P(h^s | v^s) \\ &= \max_w \sum_{s=1}^M \frac{1}{Z(v_s)} \exp \left(\sum_{k=1}^K w_k F_k(h^s, v^s) \right) \\ &= \max_w \sum_{s=1}^M \frac{1}{Z(v_s)} \exp \left(\sum_{k=1}^K \sum_{i=2}^N w_k f_k(h_i^s, h_{i-1}^s, v^s, i) \right) \end{aligned}$$

条件随机场：训练

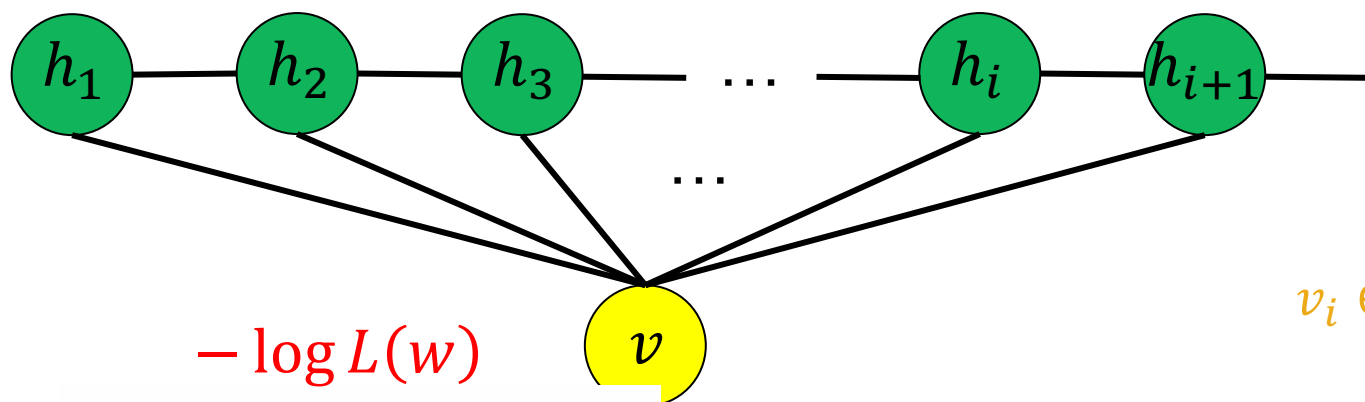


$$\max_w L(w) = \max_w \sum_{s=1}^M \frac{1}{Z(v_s)} \exp \left(\sum_{k=1}^K \sum_{i=2}^N w_k f_k(h_i^s, h_{i-1}^s, v^s, i) \right)$$

$$\Leftrightarrow \max_w [\log L(w)] = \max_w \left[\log \sum_{s=1}^M \frac{1}{Z(v_s)} \exp \left(\sum_{k=1}^K \sum_{i=2}^N w_k f_k(h_i^s, h_{i-1}^s, v^s, i) \right) \right]$$

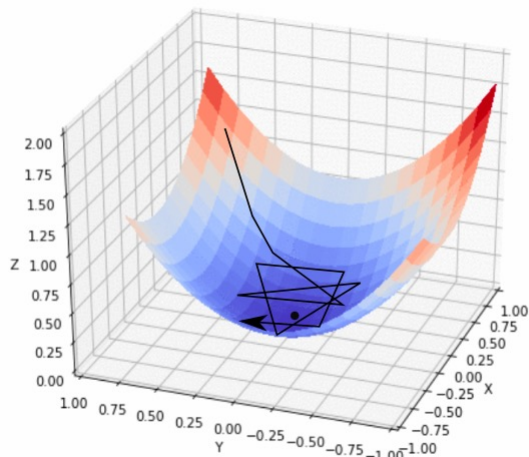
$$\Leftrightarrow \min_w [-\log L(w)] = \min_w \left[-\log \sum_{s=1}^M \frac{1}{Z(v_s)} \exp \left(\sum_{k=1}^K \sum_{i=2}^N w_k f_k(h_i^s, h_{i-1}^s, v^s, i) \right) \right]$$

条件随机场：训练



$h_i \in \begin{Bmatrix} B, M \\ E, S \end{Bmatrix}$

$v_i \in \left\{ \begin{array}{l} \text{中, 国, 人, 为,} \\ \text{了, 实, 现, 自,} \\ \text{己, 的, 梦, 想, ...} \end{array} \right\}$



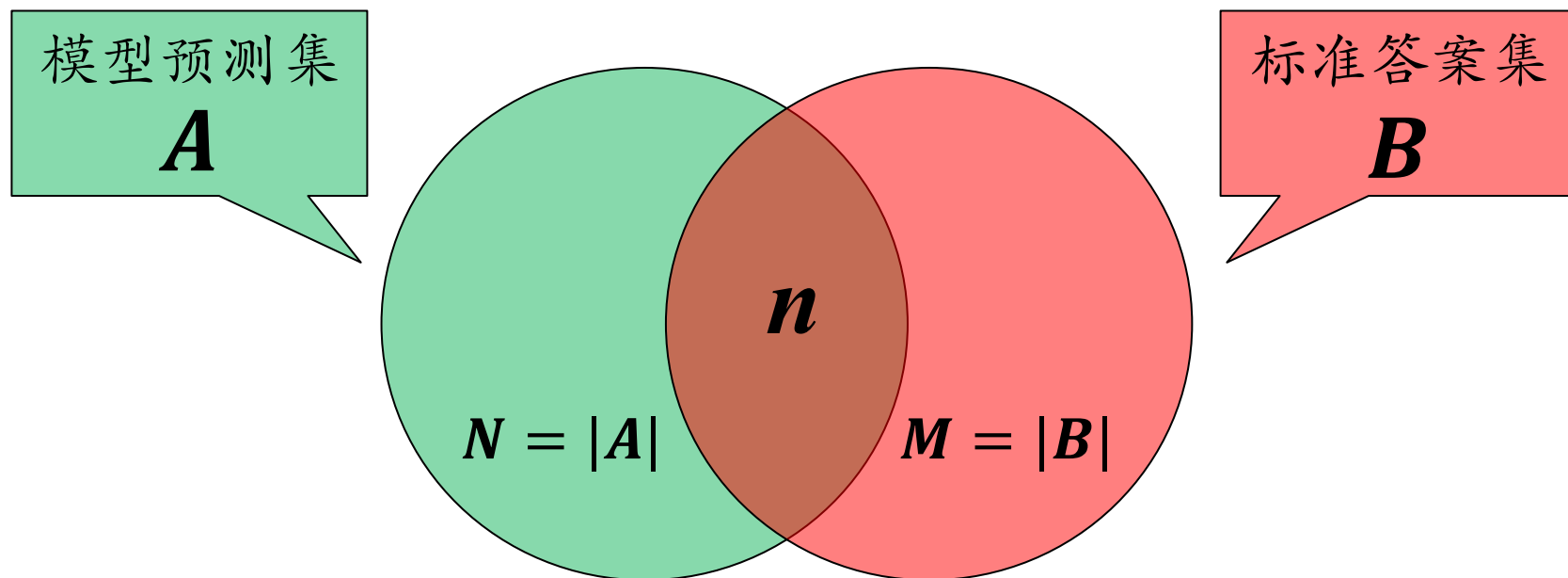
$$w^{p+1} = w^p - \eta \frac{\partial [-\log L(w)]}{\partial w}$$

$$\Leftrightarrow \min_w [-\log L(w)] = \min_w \left[-\log \sum_{s=1}^M \frac{1}{Z(v_s)} \exp \left(\sum_{k=1}^K \sum_{i=2}^N w_k f_k(h_i^s, h_{i-1}^s, v^s, i) \right) \right]$$

- ▶ 什么是中文分词
- ▶ 中文分词的几个主要算法
 - ▶ 最大匹配法
 - ▶ 最短路径法
 - ▶ 语言模型法
 - ▶ 条件随机场
- ▶ 条件随机场 (CRF) 的几个重要算法
 - ▶ 给定文本计算给定分词方案出现的概率：前向算法
 - ▶ 给定文本推断分词方案：Viterbi算法
 - ▶ 有监督学习：最大似然参数估计
- ▶ 中文分词的评价指标

中文分词的评价指标：查准率与查全率

假设系统输出 N 个结果，其中，正确的结果为 n 个，标准答案的个数为 M 个



Precision:

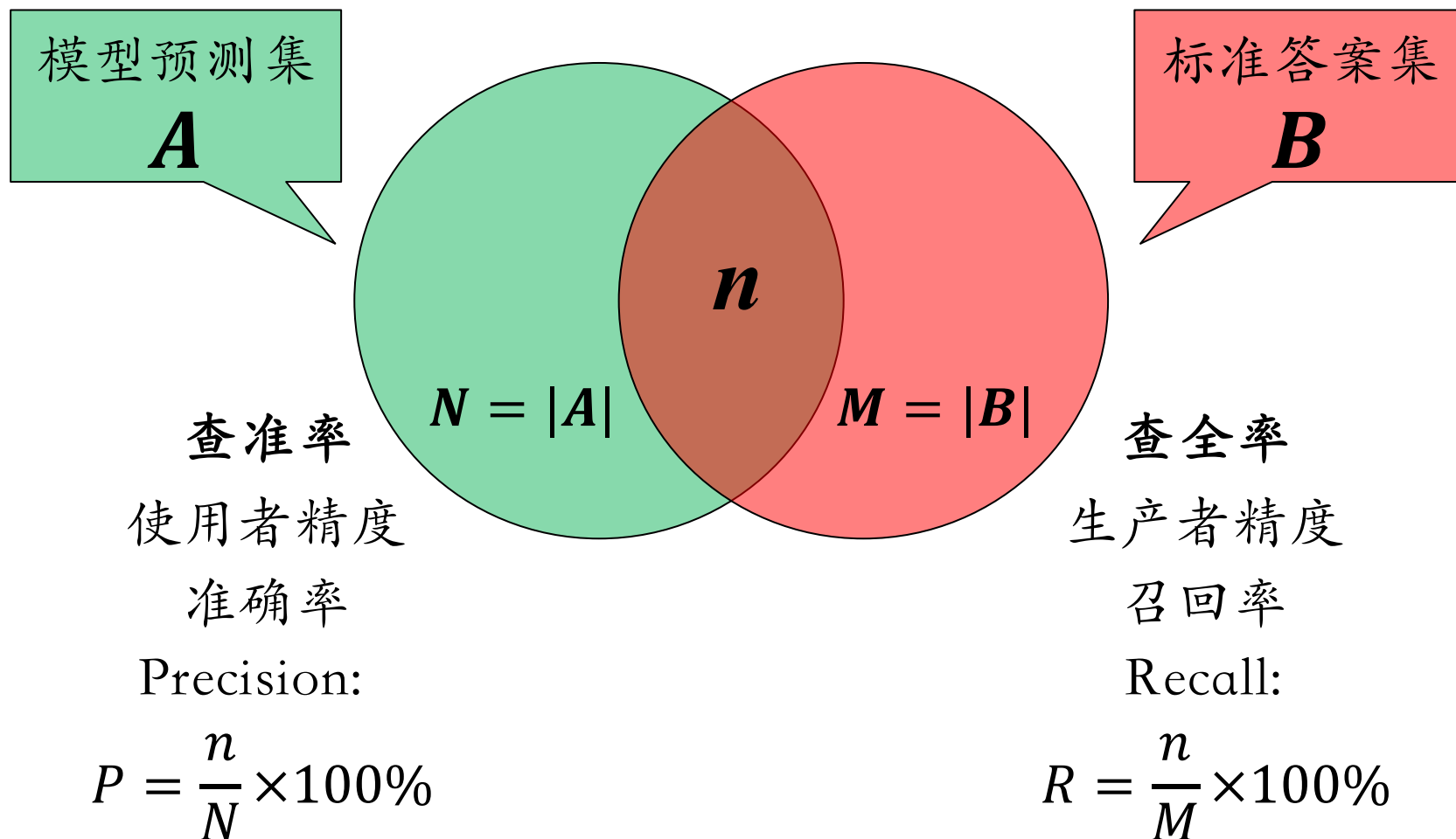
$$P = \frac{n}{N} \times 100\%$$

Recall:

$$R = \frac{n}{M} \times 100\%$$

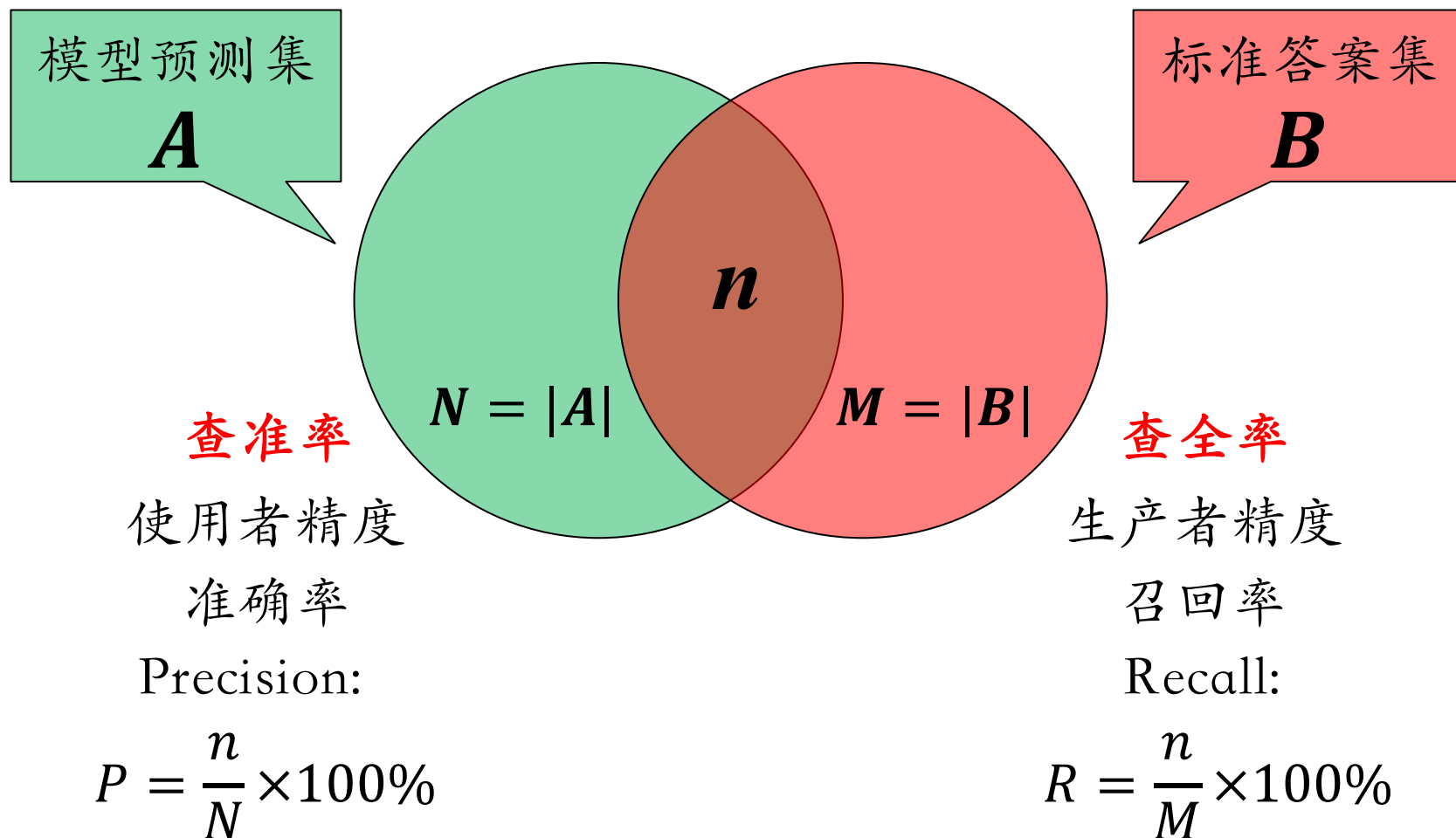
中文分词的评价指标：查准率与查全率

假设系统输出N个结果，其中，正确的结果为n个，标准答案的个数为M个



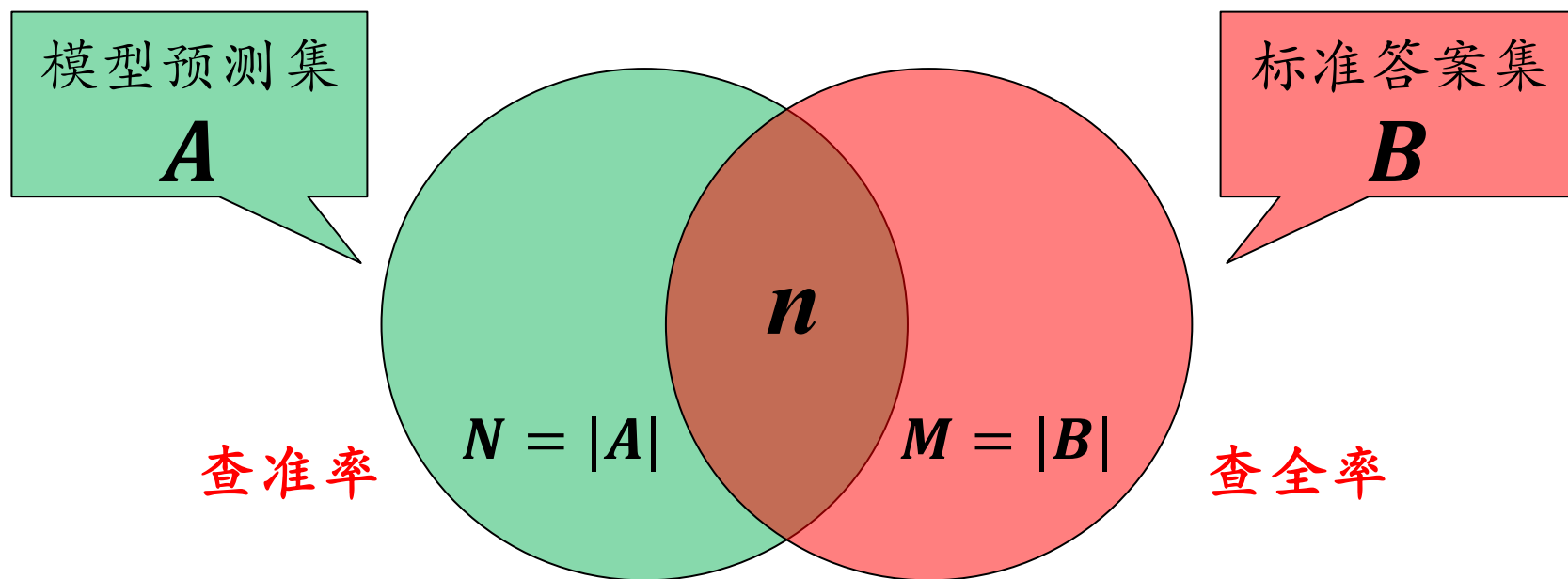
中文分词的评价指标：查准率与查全率

假设系统输出N个结果，其中，正确的结果为n个，标准答案的个数为M个



中文分词的评价指标：F-measure

假设系统输出N个结果，其中，正确的结果为n个，标准答案的个数为M个



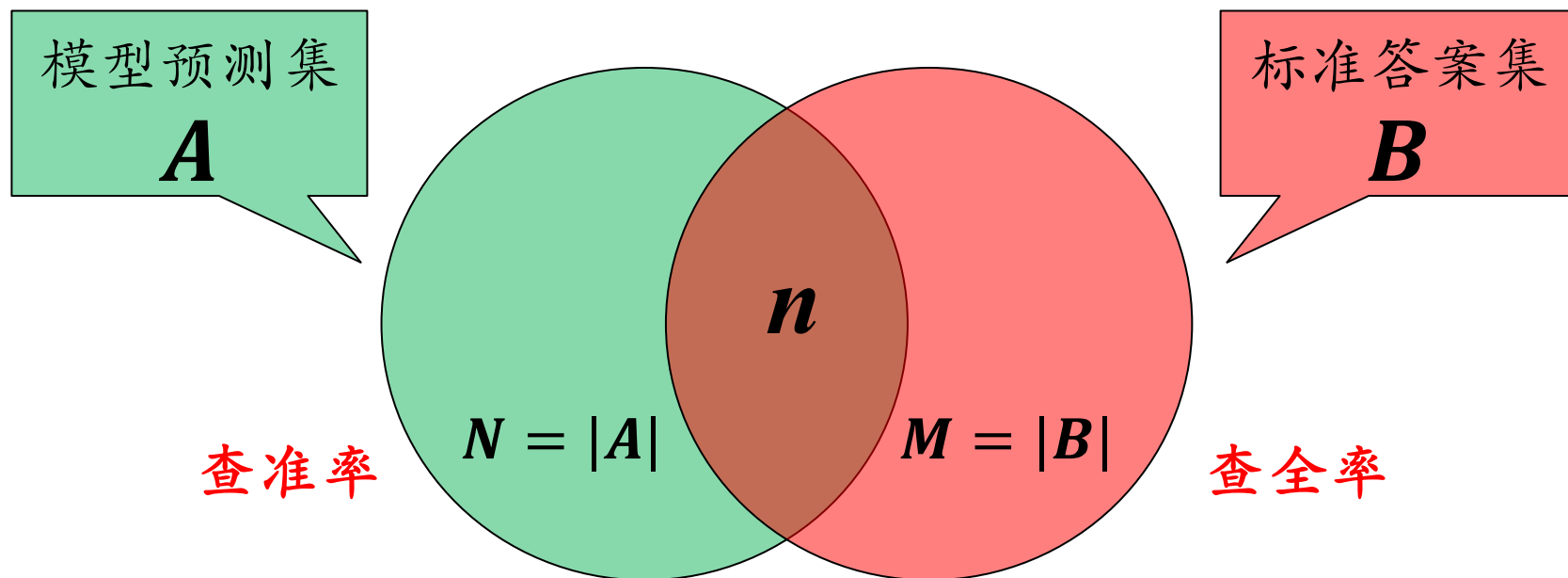
$$F_{\beta} = \frac{(1 + \beta^2)PR}{\beta^2 P + R} \times 100\%$$

$$P = \frac{n}{N} \times 100\%$$

$$R = \frac{n}{M} \times 100\%$$

中文分词的评价指标：F-measure

假设系统输出N个结果，其中，正确的结果为n个，标准答案的个数为M个



$$F_1 = \frac{2PR}{P + R} \times 100\%$$

$$P = \frac{n}{N} \times 100\%$$

$$R = \frac{n}{M} \times 100\%$$

中文分词的评价指标：小测试

假设某个汉语分词系统在一测试集上输出 5260 个分词结果，而标准答案是 4510 个词语，根据这个答案，系统切分出来的结果中有 4120 个是正确的。那么：

$$P = \frac{4120}{5260} \times 100\% = 78.33\%$$

$$R = \frac{4120}{4510} \times 100\% = 91.35\%$$

$$F_1 = \frac{2 \times 78.33\% \times 91.35\%}{78.33\% + 91.35\%} \times 100\% = 84.34\%$$

中文分词：一些实战经验

以SIGHAN Bakeoff 评测语料(2005)为例：

语言模型法 (3-gram)

P=89.8%

CRF

P=94.3%

实用工具：CRF++

- ▶ 英文分词与词元切分 (tokenization)
 - ▶ 问题与需求
 - ▶ 词级别的词元切分 (word-level tokenization)
 - ▶ 基于正则表达式的词元切分
 - ▶ 亚词级别的词元切分 (subword-level tokenization)
 - ▶ BPE: byte pair encoding
- ▶ 常用工具

- ▶ 英文分词与词元切分 (tokenization)
 - ▶ 问题与需求
 - ▶ 词级别的词元切分 (word-level tokenization)
 - ▶ 基于正则表达式的词元切分
 - ▶ 亚词级别的词元切分 (subword-level tokenization)
 - ▶ BPE: byte pair encoding
- ▶ 常用工具

简单句子的切分

You are what you eat.

From the above experiments we conclude that for sentence splitting the corpus being used for training is not so critical because sentence annotations are not really controversial within different biomedical subdomains.

简单按照空格和标点符号切分即可

I'm right, aren't I?

切分方法一：将缩写后的词作为一个新词

I'm / right / , / aren't / I / ?

can't, don't, couldn't, shouldn't, wouldn't...

切分方法二：将缩写后的词拆开

I / 'm / right / , / are / n't / I / ?

O'neil, o'clock, O'reilly...

The total revenue rises above \$300,000,000. (c.f. Fig. 3)

切分方法一：将数字和符号按照标点切开

\$ / 300 / , / 000 / , / 000

Fig. / 3

失去了数字的意义

Fig-3; Table-2

切分方法二：不拆开

\$300,000,000

Fig. 3

\$300,000,001;

Fig. 1; Fig. 2

\$300,000,002;

Table 3; Chart 4

\$300,000,003;

...

...

- ▶ 英文分词与词元切分 (tokenization)
 - ▶ 问题与需求
 - ▶ 词级别的词元切分 (word-level tokenization)
 - ▶ 基于正则表达式的词元切分
 - ▶ 亚词级别的词元切分 (subword-level tokenization)
 - ▶ BPE: byte pair encoding
- ▶ 常用工具

词级别的词元切分 (word-level tokenization)



基于规则的方法

- 使用正则表达式做匹配，不断修改正则表达式的数量和形式，使得在给定语料上的切分效果趋于完善

基于学习的方法

- 使用序列标注模型，在给定语料上像学习词性标注与中文分词那样学习词元的切分。

词级别的词元切分 (word-level tokenization)

```
# starting quotes
STARTING_QUOTES = [
    (re.compile(r"^\""), r"``"),
    (re.compile(r"(`)"), r" \1 "),
    (re.compile(r"([\{\<])\"|\'{2}"), r"\1 `` "), ]

# punctuation
PUNCTUATION = [
    (re.compile(r"([:,])([^\d])"), r" \1 \2"),
    (re.compile(r"([:,])$"), r" \1 "),
    (re.compile(r"\.\.\."), r" ... "),
    (re.compile(r"[;@#%&]"), r" \g<0> "),
    (re.compile(r'([^\.])([^\.])([^\.]>"\' ]*)\s*$'), r"\1 \2\3 ", ), # Handles the final period.
    (re.compile(r"[?!]"), r" \g<0> "),
    (re.compile(r"([\' ]' ") , r"\1 ' "), ]

# Pads parentheses
PARENS_BRACKETS = (re.compile(r"[\]\[\(\)\{\}\<\>]"), r" \g<0> ")

# Optionally: Convert parentheses, brackets and converts them to PTB symbols.
CONVERT_PARENTHESES = [
    (re.compile(r"\("), "-LRB-"),
    (re.compile(r"\)"), "-RRB-"),
    (re.compile(r"\["), "-LSB-"),
    (re.compile(r"\]"), "-RSB-"),
    (re.compile(r"\{"), "-LCB-"),
    (re.compile(r"\}"), "-RCB-"), ]

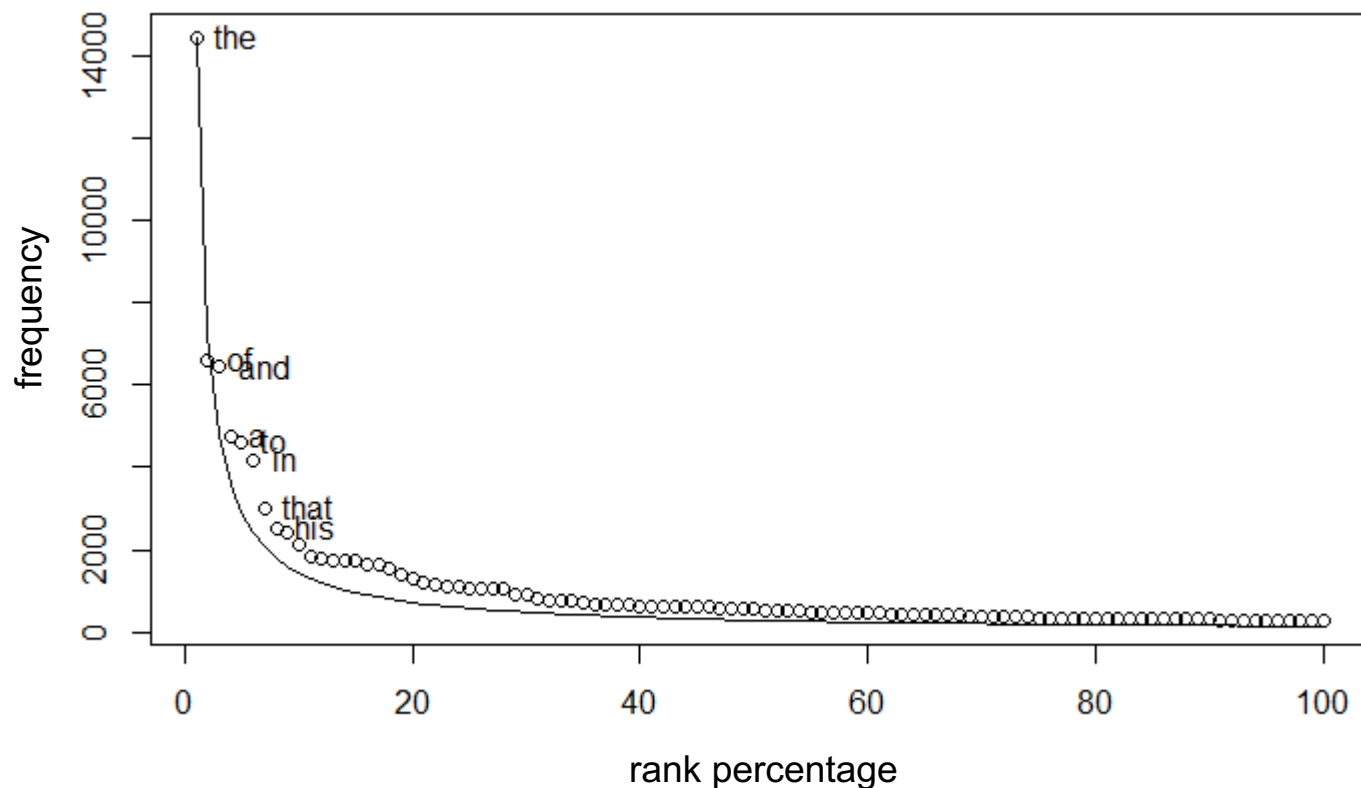
DOUBLE_DASHES = (re.compile(r"--"), r" -- ")

# ending quotes
ENDING_QUOTES = [
    (re.compile(r'\''), " '' "),
    (re.compile(r"(\S)(\')"), r"\1 \2 "),
    (re.compile(r"([\' ])(('[sS] | '[mM] | '[dD] | ' ) )"), r"\1 \2 "),
    (re.compile(r"([\' ])('ll | 'LL | 're | 'RE | 've | 'VE | n't | N'T )"), r"\1 \2 "), ]
```

- ▶ 英文分词与词元切分 (tokenization)
 - ▶ 问题与需求
 - ▶ 词级别的词元切分 (word-level tokenization)
 - ▶ 基于正则表达式的词元切分
 - ▶ 亚词级别的词元切分 (subword-level tokenization)
 - ▶ BPE: byte pair encoding
- ▶ 常用工具

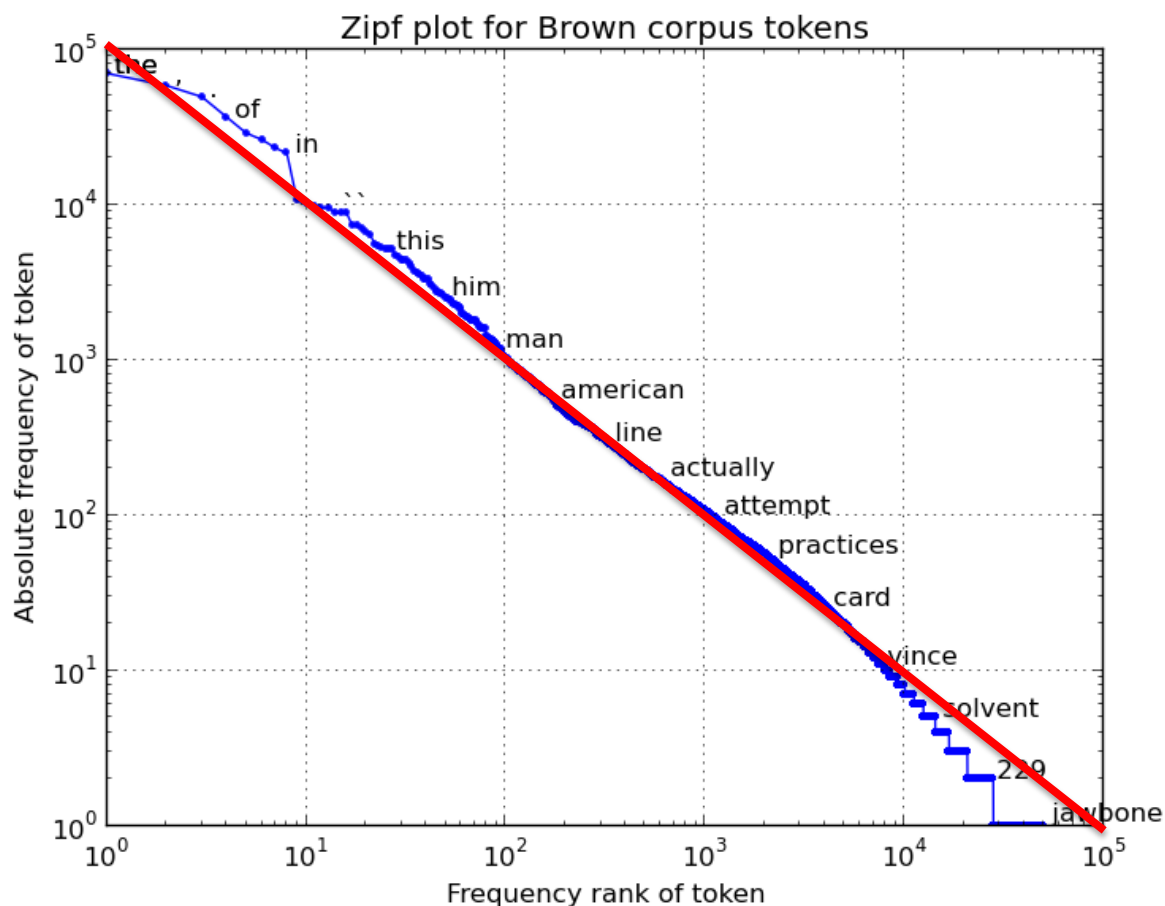
亚词级别的词元切分 (subword-level tokenization)

对于英语、法语、德语等语言而言，词并不是一个特别合适的切分单元



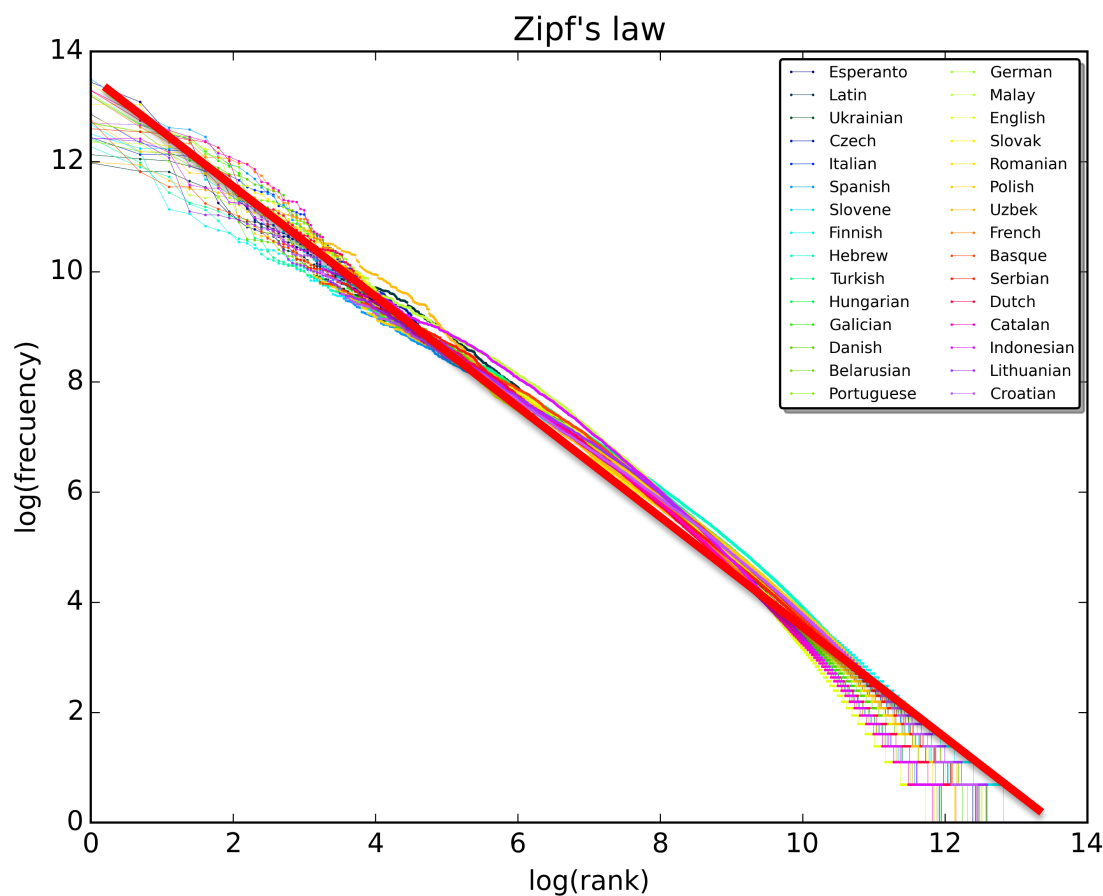
亚词级别的词元切分 (subword-level tokenization)

如果我们使用双对数轴线:

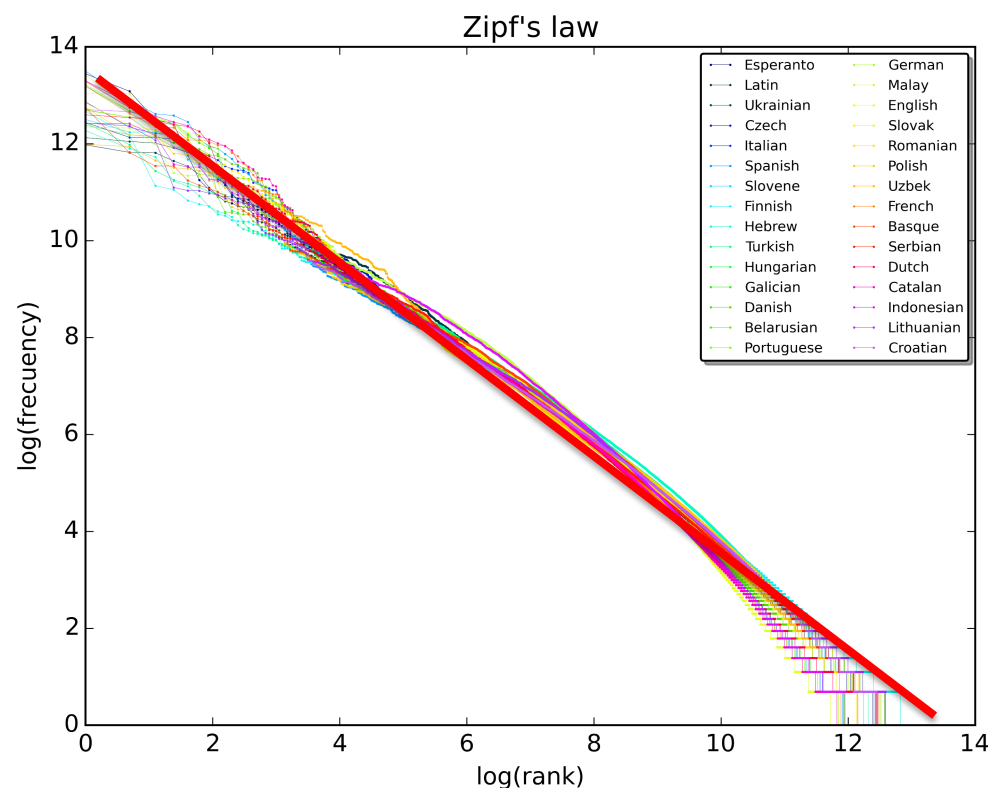


亚词级别的词元切分 (subword-level tokenization)

同样的情况广泛存在于各种语言。比如对30种语言的维基百科进行词频统计：



亚词级别的词元切分 (subword-level tokenization)



总的来说，几乎所有语言中词频的分布都大致符合

“Zipf's Law” ,

即对于包含有N个不同单词的语料集，其第k个最常见单词归一化频度为：

$$f(k) = \frac{1}{k^s} \bigg/ \sum_{i=1}^N \frac{1}{i^s}$$

其中s为参数，随具体语言的不同而不同。

这样的分布对应到双对数轴线上，就是一条直线。

亚词级别的词元切分 (subword-level tokenization)

对于英语、法语、德语等语言而言，词并不是一个特别合适的切分单元

- 对于一个中等到大型的语料集而言，其涉及到的词汇将达到30~60万。
- 即便刨除只出现一两次的罕见词，词汇表仍可轻松超越10万。且大量的未登录词会造成模型在具体任务上性能的下降。

然而大量的单词之间存在互相联系，或是同一个词的不同屈折变化，或是共享部分词根

亚词级别的词元切分 (subword-level tokenization)

对于英语、法语、德语等语言而言，词并不是一个特别合适的切分单元

- 同一个词的不同屈折变化
- 不同词之间的联系

例如：

response
responsive
responsible
responsibility
...

例如：

contract
contradictory
contrastive
inventive
responsive
bilateral
bicycle
reserve
preserve
conserve
...

亚词级别的词元切分 (subword-level tokenization)

对于英语、法语、德语等语言而言，词并不是一个特别合适的切分单元

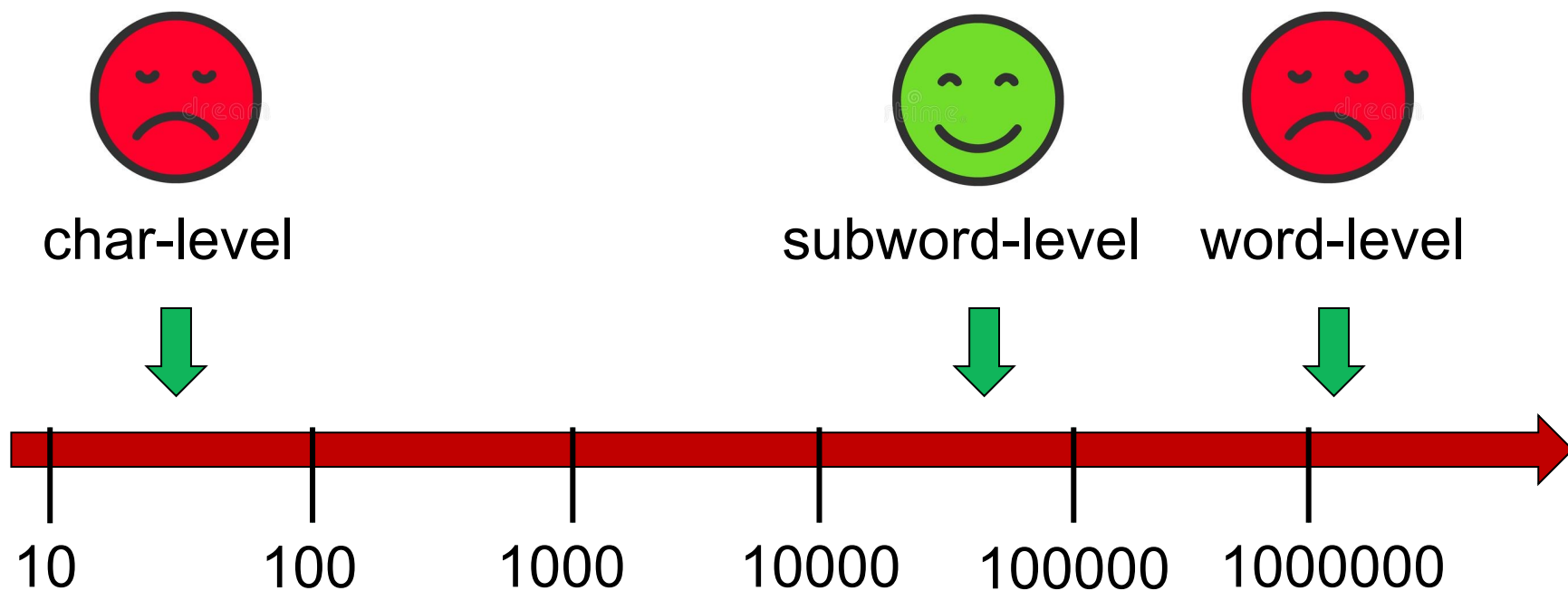
然而，除了词之外，字母也不是很好的切分单元。

- 各个语言，主要是英语等欧洲语言中，字母表的数目在几十的量级，即便考虑某些语言的accent（如àáâãäåéèëïòö等）字母表也很难超过100。

亚词级别的词元切分 (subword-level tokenization)

对于英语、法语、德语等语言而言，词并不是一个特别合适的切分单元

然而，除了词之外，字母也不是很好的切分单元。



BPE: Byte-Pair Encoding

BPE是2016年提出的亚词级别词元切分方法[1]，其使用的算法直接继承自1994年的数据压缩领域的论文[2]。

该算法从数据中自底向上地组合字母来形成词元，词元数目精确可控。

通过适当调整参数，可以得到类似于“词根词缀”的英文亚词单元。

1. Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units." In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1715-1725. 2016.
2. Gage, Philip. "A new algorithm for data compression." The C Users Journal 12, no. 2 (1994): 23-38.

BPE: Byte-Pair Encoding

Byte pair encoding or digram coding is a simple form of data compression in which the most common pair of consecutive bytes of data is replaced with a byte that does not occur within that data. A table of the replacements is required to rebuild the original data. The algorithm was first described publicly by Philip Gage in a February 1994 article “A New Algorithm for Data Compression” in the C Users Journal. A variant of the technique has shown to be useful in several natural language processing (NLP) applications, such as Google’s SentencePiece, and OpenAI’s GPT-3.

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6}

1. 初始词汇表：将所有出现的单个字母全都包含进来

l o w e s t n r i d _

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6}

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token

l o w e s t n r i d _ +

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6 }

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token

l o w e s t n r i d _ + r _

BPE: Byte-Pair Encoding

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6 }

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token

l o w e s t n r i d _ + e r _

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6 }

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token

lowestnr_id_ + er_

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6 }

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token

l o w e s t n r i d _ + e r _ l o

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6 }

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token

lowestnr_id_ + er_ low

BPE: Byte-Pair Encoding: 算例

{ 'low_':5, 'lowest_':2, 'newer_':3, 'wider_':6}

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token
3. 终止：迭代达到预先设定的最大次数时停止

最终词汇表中的词元数目为：初始词汇数+迭代次数

l o w e s t n r i d _ + e r _ l o w

BPE: Byte-Pair Encoding: 算法流程

1. 初始词汇表：将所有出现的单个字母全都包含进来
2. 迭代：
 - (1) 将现有语料库用现有词汇表表示；
 - (2) 再将其中最常见的一个bi-gram捏合成一个新的token
3. 终止：迭代达到预先设定的最大次数时停止

最终词汇表中的词元数目为：初始词汇数+迭代次数

BPE: Byte-Pair Encoding: 优点

1. 解决了字母级与单词级的词元切分各自的问题，且词元数量精确可控。
2. 解决了未登录词的问题：无论设置的词元数量是多少，总是可以有办法使用有限的词元覆盖所有可能出现的单词。
3. 算法复杂度低，可以在大规模语料上实用。
4. 是目前绝大部分NLP模型采用的词元切分方法。

中英文对比

中文

你是个好人。



你/是个/好人/。

你/是/个/好/人/。

丶 一 | 丿 ……

汉字



笔画

句子



单词



≈

亚词



字母

≈

英文

You are a good person.



You/are/a/good/person/.



You_/are_/a_/good_/p
er/son_/.

abcdefg…

- ▶ 英文分词与词元切分 (tokenization)
 - ▶ 问题与需求
 - ▶ 词级别的词元切分 (word-level tokenization)
 - ▶ 基于正则表达式的词元切分
 - ▶ 亚词级别的词元切分 (subword-level tokenization)
 - ▶ BPE: byte pair encoding
- ▶ 常用工具

词元切分：常用工具

词级别的词元切分 (word-level tokenization)

序号	库名/函数名	描述
1	<code>nltk.tokenize.word_tokenize</code>	基于正则表达式的词级别词元切分
2	<code>nltk.tokenize.wordpunct_tokenize</code>	基于标点的词级别词元切分
3	<code>spacy.tokenizer.Tokenizer</code>	基于正则表达式的词级别词元切分
4	<code>mosestokenizer</code>	基于正则表达式的词级别词元切分

亚词级别的词元切分 (subword-level tokenization)

序号	库名/函数名	描述
1	<code>sentencepiece</code>	使用BPE从句子直接切分出亚词
2	<code>wordpiece</code>	使用BPE从词切分出亚词

Vocabulary Learning via Optimal Transport for Neural Machine Translation

Jingjing Xu¹, Hao Zhou¹, Chun Gan^{1,2†}, Zaixiang Zheng^{1,3†}, Lei Li¹

¹ByteDance AI Lab

²Math Department, University of Wisconsin–Madison

³Nanjing University

{xujingjing.melody, zhouhao.nlp, lileilab}@bytedance.com

cgan5@wisc.edu

zhengzx@smail.nju.edu.cn

Abstract

The choice of token vocabulary affects the performance of machine translation. This paper aims to figure out what is a good vocabulary and whether one can find the optimal vocabulary without trial training. To answer these questions, we first provide an alternative understanding of the role of vocabulary from the perspective of information theory. Motivated by this, we formulate the quest of vocabularization – finding the best token dictionary with a proper size – as an optimal transport (OT) problem. We propose **VOLT**, a simple and efficient solution without trial training. Empirical results show that VOLT outperforms widely-used vocabularies in diverse scenarios, including WMT-14 English-German and TED multilingual translation. For example, VOLT achieves almost 70% vocabulary size reduction and 0.5 BLEU gain on English-German translation. Also, compared to BPE-search, VOLT reduces the search time from 384 GPU

2018; Al-Rfou et al., 2019; Wang et al., 2020).

The key idea of these approaches is selecting the most frequent sub-words (or word pieces with higher probabilities) as the vocabulary tokens. In information theory, these frequency-based approaches are simple forms of data compression to reduce entropy (Gage, 1994), which makes the resulting corpus easy to learn and predict (Martin and England, 2011; Bentz and Alikaniotis, 2016).

However, the effects of vocabulary size are not sufficiently taken into account since current approaches only consider frequency (or entropy) as the main criteria. Many previous studies (Sennrich and Zhang, 2019; Ding et al., 2019; Provilkov et al., 2020; Salesky et al., 2020) show that vocabulary size also affects downstream performances, especially on low-resource tasks. Due to the lack of appropriate inductive bias about size, trial training (namely traversing all possible sizes) is usually required to search for the optimal size, which takes